

Analysis and Design of Algorithms

CIE 3 — Lecture Notes

Section 1: Space and Time Trade-offs

Section 2: Greedy Technique

Section 3: Backtracking

Section 4: Branch and Bound

Section 5: NP-Completeness

Harsha Matadhikari

Professor of Practice

Dept. of Computer Science and Engineering

B.M.S. College of Engineering, Bengaluru

4th Semester — Feb–May 2026

Based on: Levitin, Introduction to the Design and Analysis of Algorithms, 3rd ed.

Syllabus Coverage — CIE 3

Ch.	Design Strategy	Representative Algorithms
1	Space & Time Trade-offs	Horspool's algorithm, Boyer-Moore algorithm
2	Greedy Technique	Fractional knapsack, Prim's MST, Kruskal's MST, Dijkstra's shortest path, Huffman coding
3	Backtracking	n -Queens, Subset-sum
4	Branch and Bound	0/1 Knapsack (B&B), Travelling Salesman Problem
5	NP-Completeness	Complexity classes P and NP, Cook's theorem, polynomial reductions, Clique, Vertex Cover

Contents

Syllabus Coverage — CIE 3	2
1 Space and Time Trade-offs	5
1.1 Strategy Overview	5
1.2 Horspool’s Algorithm	6
1.3 Boyer-Moore Algorithm	8
1.4 Worked Example: Pattern YALA in MALAYALAM_ALAYAM	10
1.5 Worked Example: Pattern LAYAM in MALAYALAM_ALAYAM	12
1.6 Comparison of String-Matching Algorithms	15
1.7 End-of-Chapter Questions	15
2 Greedy Technique	16
2.1 Strategy Overview	16
2.2 Fractional Knapsack	17
2.3 Prim’s Algorithm (Minimum Spanning Tree)	19
2.4 Kruskal’s Algorithm (Minimum Spanning Tree)	21
2.5 Dijkstra’s Algorithm — Single-Source Shortest Paths	24
2.6 Huffman Coding	26
2.7 End-of-Chapter Questions	29
3 Backtracking	31
3.1 Strategy Overview	31
3.2 n -Queens Problem	32
3.3 Subset-Sum Problem	35
3.4 End-of-Chapter Questions	39
4 Branch and Bound	40
4.1 Strategy Overview	40
4.2 From Feasibility to Optimality: Why These Problems Need a Bound	42
4.3 0/1 Knapsack via Branch and Bound	43
4.4 Travelling Salesman Problem (B&B)	47
4.5 End-of-Chapter Questions	51
5 NP-Completeness	52
5.1 Motivation: Why Some Problems Resist Efficient Algorithms	52
5.2 Complexity Classes P and NP	52
5.3 NP-Completeness and Cook’s Theorem	56
5.4 NP-Complete Problems	58

5.5	End-of-Chapter Questions	62
	Note: Two Routes to a Vertex Cover	63
A	Appendix: Core Theorems and Algorithm Reference	64
A.1	Core Theorems	64
A.2	Complexity Summary: All CIE-3 Algorithms	67
A.3	Comparison of Paradigms: CIE-3	67
A.4	Supplementary: Reductions & Extra Examples (beyond the CIE-3 syllabus)	67
A.5	Extra Question Bank	71

Chapter 1

Space and Time Trade-offs

Chapter Outcomes

You will explain how precomputed shift tables trade storage for speed in string matching, build the Horspool shift table and trace the algorithm step by step, apply Boyer-Moore's two-shift heuristics (bad-character and good-suffix), and compare the three string-matching strategies on complexity and practical behaviour.

1.1 Strategy Overview

A **space-and-time trade-off** algorithm spends extra memory during a *preprocessing* phase to compute auxiliary data structures, so that the main (repeated or expensive) phase runs faster.

	Naive approach	Space-time trade-off
Pre-computation	None	Build a table / index in $O(f(\text{pattern}))$ time
Main computation	$O(nm)$ per query	$O(g(n, m))$ per query, $g \ll nm$ on average
Extra space	$O(1)$	$O(m)$ or $O(\Sigma)$

Representative algorithms in this chapter:

- **Horspool's Algorithm** — single shift table (bad-character only).
- **Boyer-Moore Algorithm** — two shift tables (bad-character + good-suffix), achieving sub-linear average-case performance.

Real-World Applications

- **Text editors:** Find/replace in large documents (grep, vim search).
- **Bioinformatics:** DNA/protein sequence search in genome databases.
- **Intrusion detection:** Network packets scanned for malware signatures.
- **Search engines:** Index-based keyword lookup.

- **Data compression:** Pattern matching in LZ77/LZ78 sliding-window coders.

1.2 Horspool's Algorithm

Key Idea

Given pattern $P[0..m-1]$ and text $T[0..n-1]$, the **naïve** algorithm slides the pattern one position at a time, giving $O(nm)$ worst-case comparisons.

Horspool's idea: instead of shifting by 1, precompute a **shift table** from the pattern that allows larger, character-guided shifts. Matching is done *right-to-left* within each window (compare $P[m-1]$ with $T[k]$ first); the shift is determined by the *current rightmost text character* $T[k]$.

Shift Table

For pattern $P[0..m-1]$ over alphabet Σ , define

$$\text{ShiftTable}[c] = \begin{cases} m - 1 - i & \text{if } c \text{ occurs in } P[0..m-2] \text{ with rightmost index } i, \\ m & \text{if } c \notin P[0..m-2]. \end{cases}$$

Algorithm 1: BuildShiftTable($P[0..m-1]$)

```

1 for each  $c \in \Sigma$  do
2   |  $\text{ShiftTable}[c] \leftarrow m$ 
3 end
4 for  $j \leftarrow 0$  to  $m-2$  do
5   |  $\text{ShiftTable}[P[j]] \leftarrow m - 1 - j$ 
6 end
7 return  $\text{ShiftTable}$ 
```

Algorithm

Algorithm 2: HorspoolMatching($P[0..m-1]$, $T[0..n-1]$)

```

1  $\text{ShiftTable} \leftarrow \text{BuildShiftTable}(P)$ 
2  $k \leftarrow m - 1$ 
3 while  $k \leq n - 1$  do
4   |  $j \leftarrow m - 1$ ;  $i \leftarrow k$ 
5   | while  $j \geq 0$  and  $P[j] = T[i]$  do
6   |   |  $j \leftarrow j - 1$ ;  $i \leftarrow i - 1$ 
7   | end
8   | if  $j < 0$  then report match at position  $i + 1$ 
9   |  $k \leftarrow k + \text{ShiftTable}[T[k]]$ 
10 end
```

Key point: The shift depends on $T[k]$ (the rightmost character of the current window), *regardless* of where the mismatch occurred. The table is computed once in $\Theta(m + |\Sigma|)$ time and reused for every window.

Trace Example 1 (Levitin §7.2)

Pattern: BARBER ($m = 6$). **Text:** JIM_SAW_ME_IN_A_BARBERSHOP ($n = 26$).

Step 1 — Build Shift Table. $P[0..4] = \text{BARBE}$ (first $m-1$ characters):

Character	A	B	E	R	all others
Rightmost index in $P[0..4]$	1	3	4	2	—
Shift = $m-1-i$	4	2	1	3	6

Step 2 — Match (right-to-left comparisons per window).

Step	k	Window $T[k-5..k]$	$T[k]$	Shift	Comparisons
1	5	JIM_SA	A	4	$P[5]=R \neq T[5]=A$ — shift
2	9	SAW_ME	E	1	$P[5]=R \neq T[9]=E$ — shift
3	10	AW_ME_	_	6	$P[5]=R \neq T[10]=_$ — shift
4	16	IN_A_B	B	2	$P[5]=R \neq T[16]=B$ — shift
5	18	_A_BAR	R	3	$P[5]=R=T[18]\checkmark$, $P[4]=E \neq T[17]=A$ — shift
6	21	BARBER	R	—	All 6 match — MATCH at position 16 $\checkmark$

Window alignment at each step:

```

Text:  J I M _ S A W _ M E _ I N _ A _ B A R B E R S H O P
Step 1: B A R B E R
Step 2:           B A R B E R
Step 3:           B A R B E R
Step 4:           B A R B E R
Step 5:           B A R B E R
Step 6:           B A R B E R ← match

```

Trace Example 2

Pattern: ABCB ($m = 4$). **Text:** ABCABCB ($n = 7$).

Shift Table ($P[0..2]=\text{ABC}$):

Character	A	B	C	others
Rightmost index	0	1	2	—
Shift	3	2	1	4

Matching trace:

Step	k	Window $T[k-3..k]$	$T[k]$	Shift	Comparisons
1	3	ABCA	A	3	$P[3]=B \neq T[3]=A$ — shift
2	6	ABCB	B	—	All 4 match — MATCH at position 3 ✓

Pattern found at position 3 in just **2 window alignments**.

Complexity: Horspool's Algorithm

Phase	Best	Average	Worst
Preprocessing (shift table)	$\Theta(m + \Sigma)$	$\Theta(m + \Sigma)$	$\Theta(m + \Sigma)$
Matching	$\Theta(n/m)$	$\Theta(n/m)$	$O(nm)$
Total	$\Theta(n/m)$	$\Theta(n/m)$	$O(nm)$

Best/Average: When the last character of the window rarely appears in the pattern, every window shifts by m characters — giving $\Theta(n/m)$ comparisons: *sub-linear*.

Worst: Pattern AAA in text AAAA...A — shift table always gives 1.

Space: $\Theta(|\Sigma|)$ for the shift table.

1.3 Boyer-Moore Algorithm

Key Idea

Boyer-Moore extends Horspool's single-table approach with a **second shift table**, giving larger guaranteed shifts and better worst-case behaviour in practice.

Rule	Trigger	Shift size
Bad-character (t_1)	Any mismatch — based on the mismatched text character	Up to m
Good-suffix (t_2)	Some suffix of window was matched — realign that suffix	Up to m
Boyer-Moore shift		$\max(t_1[\cdot], t_2[\cdot])$

Both tables are built in $\Theta(m + |\Sigma|)$ preprocessing time. Matching is right-to-left within each window, identical to Horspool.

Bad-Character Shift Table t_1

Identical to Horspool's shift table:

$$t_1[c] = \begin{cases} m - 1 - i & c \text{ in } P[0..m-2] \text{ with rightmost index } i, \\ m & c \notin P[0..m-2]. \end{cases}$$

At a mismatch at pattern position j with text character $c = T[k - (m - 1 - j)]$:

$$d_1 = \max(1, t_1[c] - (m - 1 - j)).$$

Good-Suffix Shift Table t_2

When k_{match} characters have been matched from the right (suffix $P[m - k_{\text{match}}..m - 1]$ matches $T[k - k_{\text{match}} + 1..k]$), find the largest shift $d > 0$ such that:

- the matched suffix realigns with another occurrence in P , **or**
- the longest *prefix* of P that equals a *suffix* of the matched portion can be repositioned.

Algorithm 3: BoyerMooreMatching($P[0..m-1]$, $T[0..n-1]$)

```

1 Build  $t_1$  (bad-character table) and  $t_2$  (good-suffix table) from  $P$ 
2  $k \leftarrow m - 1$ 
3 while  $k \leq n - 1$  do
4    $j \leftarrow m - 1$ ;  $i \leftarrow k$ 
5   while  $j \geq 0$  and  $P[j] = T[i]$  do
6      $j \leftarrow j - 1$ ;  $i \leftarrow i - 1$ 
7   end
8   if  $j < 0$  then report match at position  $i + 1$ 
9    $k_m \leftarrow m - 1 - j$  // number of characters matched
10  if  $k_m = 0$  then
11     $k \leftarrow k + t_1[T[k]]$ 
12  else
13     $k \leftarrow k + \max(t_2[k_m], d_1)$ 
14  end
15 end

```

Trace Example (Levitin §7.3 style)

Pattern: ABCBC ($m = 5$). **Text:** CBCABCBC ($n = 8$).

Bad-character table ($P[0..3]=ABCB$):

Character	A	B	C	others
t_1	4	1	2	5

Good-suffix table (suffix length k_m):

k_m	1	2	3	4
t_2	2	2	5	5

Derivation: Suffix “C” ($k_m=1$) reappears at $P[2]$: shift = $4-2 = 2$. Suffix “BC” ($k_m=2$) reappears at $P[1..2]$: shift = $3-1 = 2$. Suffixes of length ≥ 3 have no earlier occurrence and no matching prefix: shift = $m = 5$.

Matching trace:

Step	k	Window	Mismatch at j	k_m	Shift	Action
1	4	CBCAB	$j=4$ ($P[4]=C \neq T[4]=B$)	0	$t_1[B]=1$	No suffix matched; use bad-char
2	5	BCABC	$j=2$ ($P[2]=C \neq T[3]=A$)	2	$\max(t_1[A]=4-2, t_2[2]=2)=2$	“BC” matched; shift 2
3	7	ABCBC	—	5	—	Full match at position 3 ✓

Alignment view:

Text: C B C A B C B C
 Step 1: A B C B C
 Step 2: A B C B C
 Step 3: A B C B C $\leftarrow$ match at position 3

Complexity: Boyer-Moore Algorithm

Phase	Best	Average	Worst
Preprocessing ($t_1 + t_2$)	$\Theta(m + \Sigma)$	$\Theta(m + \Sigma)$	$\Theta(m + \Sigma)$
Matching	$\Theta(n/m)$	$\Theta(n/m)$	$O(nm)$
Total	$\Theta(n/m)$	$\Theta(n/m)$	$O(nm)$

Best/Average: Sub-linear — fewer than n characters of text are examined. With large alphabets (e.g., natural text), performance approaches $\Theta(n/m)$.

Worst: Theoretical $O(nm)$ for highly repetitive patterns/text; Galil’s 1979 improvement brings worst case to $\Theta(n)$.

Space: $\Theta(m + |\Sigma|)$ for both tables.

1.4 Worked Example: Pattern YALA in MALAYALAM_ALAYAM

Text: MALAYALAM_ALAYAM ($n = 16$); the underscore denotes a space.

Index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Char	M	A	L	A	Y	A	L	A	M	_	A	L	A	Y	A	M

Pattern YALA occurs once, at text position 4.

Horspool

Pattern: YALA ($m = 4$). **Shift Table** ($P[0..2]=YAL$):

Character	Y	A	L	others
Rightmost index in $P[0..2]$	0	1	2	—
Shift = $m-1-i$	3	2	1	4

Matching trace:

Step	k	Window $T[k-3..k]$	$T[k]$	Shift	Comparisons
1	3	MALA	A	2	$P[3]=A=T[3]✓$, $P[2]=L=T[2]✓$, $P[1]=A=T[1]✓$, $P[0]=Y \neq T[0]=M$ — shift
2	5	LAYA	A	2	$P[3]=A=T[5]✓$, $P[2]=L \neq T[4]=Y$ — shift
3	7	YALA	A	—	All 4 match — MATCH at position 4 ✓

Alignment view:

Text: M A L A Y A L A M _ A L A Y A M

Step 1: Y A L A

Step 2: Y A L A

Step 3: Y A L A $\leftarrow$ match at position 4

Total: 3 windows, $4 + 2 + 4 = 10$ character comparisons.

Boyer-Moore

Bad-character table t_1 (identical to Horspool's shift table):

Character	Y	A	L	others
t_1	3	2	1	4

Good-suffix table t_2 :

k_m (suffix length)	1	2	3
t_2	2	4	4

Derivation:

- $k_m = 1$: matched suffix "A" reappears at $P[1]$, preceded by $P[0]=Y \neq P[2]=L ✓$. Shift $= (m-1) - 1 = 2$.
- $k_m = 2$: matched suffix "LA" has no other occurrence; no proper prefix of P matches a suffix of "LA". Shift $= m = 4$.
- $k_m = 3$: matched suffix "ALA" has no other occurrence; no prefix match. Shift $= m = 4$.

Matching trace:

Step	k	Window	Mismatch at j	k_m	Shift	Action
1	3	MALA	$j=0$ ($P[0]=Y \neq T[0]=M$)	3	$\max(d_1, t_2[3]) = \max(1, 4) = 4$	"ALA" matched; good-suffix dominates
2	7	YALA	—	4	—	Full match at position 4 ✓

Bad-char detail at Step 1: $d_1 = \max(1, t_1[M] - (m-1-j)) = \max(1, 4 - 3) = 1$.

Alignment view:

Text: M A L A Y A L A M _ A L A Y A M

Step 1: Y A L A

Step 2: Y A L A $\leftarrow$ match at position 4

Total: 2 windows, $4 + 4 = 8$ character comparisons.

Comparison: Horspool vs Boyer-Moore on YALA

Algorithm	Windows	Character comparisons
Horspool	3	10
Boyer-Moore	2	8

Step-by-step shift comparison:

Step	k	k_m matched	Horspool shift	Boyer-Moore shift
1	3	3 ("ALA")	$t_1[T[3]]=t_1[A]=2$	$\max(1, t_2[3])=4$
2	5 (Horspool)	1 ("A")	$t_1[T[5]]=t_1[A]=2$	— (already at $k=7$)
3 / 2	7	4 (full)	match	match

Why Boyer-Moore wins here: At Step 1 three pattern characters matched ($k_m=3$) before mismatch at $j=0$. Horspool ignores the matched suffix and consults only $t_1[T[k]]=t_1[A]=2$, repeating partial work on the next window. Boyer-Moore reads $t_2[3] = m = 4$ and jumps the full pattern length, landing exactly on the match. This is the textbook case where the good-suffix rule pays off.

1.5 Worked Example: Pattern LAYAM in MALAYALAM_ALAYAM

Same text $T = \text{MALAYALAM_ALAYAM}$ ($n = 16$). Pattern LAYAM occurs once, at text position 11.

Horspool

Pattern: LAYAM ($m = 5$). Shift Table ($P[0..3]=\text{LAYA}$):

Character	L	A	Y	M	others
Rightmost index in $P[0..3]$	0	3	2	—	—
Shift	4	1	2	5	5

Note: M is the last character of P , so it is absent from $P[0..m-2]$ — shift is $m = 5$.

Matching trace:

Step	k	Window $T[k-4..k]$	$T[k]$	Shift	Comparisons
1	4	MALAY	Y	2	$P[4]=M \neq T[4]=Y$ — shift
2	6	LAYAL	L	4	$P[4]=M \neq T[6]=L$ — shift
3	10	LAM_A	A	1	$P[4]=M \neq T[10]=A$ — shift
4	11	AM_AL	L	4	$P[4]=M \neq T[11]=L$ — shift
5	15	LAYAM	M	—	All 5 match — MATCH at position 11 ✓

Alignment view:

```

Text:  M A L A Y A L A M _ A L A Y A M
Step 1: L A Y A M
Step 2:      L A Y A M
Step 3:          L A Y A M
Step 4:              L A Y A M
Step 5:                  L A Y A M ← match

```

Total: 5 windows, $1 + 1 + 1 + 1 + 5 = 9$ character comparisons.

Boyer-Moore

Bad-character table t_1 (identical to Horspool's):

Character	L	A	Y	M	others
t_1	4	1	2	5	5

Good-suffix table t_2 :

k_m	1	2	3	4
t_2	5	5	5	5

Derivation: The pattern's last character M appears nowhere else in LAYAM. Every non-empty matched suffix ends in M, so it cannot reappear inside P ; and no proper prefix of LAYAM (which begins with L) matches a suffix (which ends in M). Hence $t_2[k_m] = m = 5$ for every $k_m \in \{1, 2, 3, 4\}$.

Matching trace:

Step	k	Window	Mismatch at j	k_m	Shift	Action
1	4	MALAY	$j=4$ ($P[4]=M \neq T[4]=Y$)	0	$t_1[Y]=2$	$k_m=0$: bad-char only
2	6	LAYAL	$j=4$ ($P[4]=M \neq T[6]=L$)	0	$t_1[L]=4$	Same
3	10	LAM_A	$j=4$ ($P[4]=M \neq T[10]=A$)	0	$t_1[A]=1$	Same
4	11	AM_AL	$j=4$ ($P[4]=M \neq T[11]=L$)	0	$t_1[L]=4$	Same
5	15	LAYAM	—	5	—	Full match at position 11 ✓

Alignment view: identical to Horspool's at every step.

```

Text:  M A L A Y A L A M _ A L A Y A M
Step 1: L A Y A M
Step 2:      L A Y A M
Step 3:          L A Y A M
Step 4:              L A Y A M
Step 5:                  L A Y A M ← match

```

Total: 5 windows, $1 + 1 + 1 + 1 + 5 = 9$ character comparisons.

Comparison: Horspool vs Boyer-Moore on LAYAM

Algorithm	Windows	Character comparisons
Horspool	5	9
Boyer-Moore	5	9

Step-by-step shift comparison:

Step	k	k_m matched	Horspool shift	Boyer-Moore shift
1	4	0	$t_1[Y]=2$	$t_1[Y]=2$
2	6	0	$t_1[L]=4$	$t_1[L]=4$
3	10	0	$t_1[A]=1$	$t_1[A]=1$
4	11	0	$t_1[L]=4$	$t_1[L]=4$
5	15	5	match	match

Why Boyer-Moore gains nothing here: The pattern ends in M, a character that occurs nowhere else in LAYAM. Every window therefore fails on the very first comparison ($j=4$, $k_m=0$), and Boyer-Moore's good-suffix rule never activates. With $k_m=0$ at every step, the algorithm reduces by definition to Horspool. This illustrates that Boyer-Moore's edge requires the pattern to allow *partial right-to-left matches* before mismatch.

Lesson from the two examples

Both algorithms share the same right-to-left comparison and the same bad-character table. They diverge only when at least one character matches before a mismatch ($k_m \geq 1$).

- Pattern YALA produces a 3-character partial match in window MALA — Boyer-Moore's good-suffix table fires and saves an entire window.
- Pattern LAYAM ends in a character (M) that is unique to its position, so every window fails on the first comparison and the two algorithms behave identically. The size of Boyer-Moore's win depends entirely on the pattern's *internal structure*, not on the length of the text.

1.6 Comparison of String-Matching Algorithms

Algorithm	Direction	Tables	Best	Average	Worst
Naive	Left-to-right	None	$\Theta(n)$	$\Theta(n)$	$\Theta(nm)$
Horspool	Right-to-left	t_1 only	$\Theta(n/m)$	$\Theta(n/m)$	$O(nm)$
Boyer-Moore	Right-to-left	$t_1 + t_2$	$\Theta(n/m)$	$\Theta(n/m)$	$O(nm)$

Both Horspool and Boyer-Moore are identical in asymptotic complexity, but Boyer-Moore achieves larger shifts in practice (especially on small alphabets such as DNA) due to the good-suffix table.

1.7 End-of-Chapter Questions

Numerical Questions

1. Build Horspool's shift table for pattern ABCAB over the alphabet {A, B, C} and any other character.
2. Trace Horspool's algorithm for pattern NOT in text NOBODY_NOTICED. Show each window alignment and shift.
3. Build both t_1 and t_2 tables for pattern BAAB.
4. Trace Boyer-Moore for pattern BAAB in text AABABAAB. At each step, show both shift values and which one is used.
5. For pattern AAA and text AAAAAAAAA: show that Horspool's worst case $O(nm)$ is achieved and count the total number of character comparisons.
6. Explain why Horspool's shift is based on $T[k]$ (the rightmost window character) rather than the mismatched character $T[i]$.
7. What modification to Horspool's shift table would you make if you wanted to find all non-overlapping occurrences of a pattern?
8. Compare Horspool's algorithm with the KMP (Knuth-Morris-Pratt) algorithm in terms of shift direction, preprocessing cost, and average-case performance.

LeetCode Practice

1. 28 — Find the Index of the First Occurrence in a String (implement Horspool's)
2. 459 — Repeated Substring Pattern
3. 686 — Repeated String Match
4. 1408 — String Matching in an Array

Chapter 2

Greedy Technique

Chapter Outcomes

You will apply the greedy-choice property and optimal substructure to design greedy algorithms, prove greedy correctness via exchange arguments, trace Prim's, Kruskal's, Dijkstra's, and Huffman's algorithms step by step, and derive their time complexities.

2.1 Strategy Overview

A **greedy algorithm** builds a solution piece by piece, at each step choosing the option that looks best *locally* — and never revisiting that choice.

Two Conditions for Correctness

- **Greedy-choice property:** A globally optimal solution can be reached by making locally optimal (greedy) choices. The local choice is provably safe: selecting the best-available item now cannot block a better overall solution.
- **Optimal substructure:** After making a greedy choice, the remaining sub-problem also has optimal substructure — i.e., the Principle of Optimality holds.

Greedy vs. Dynamic Programming

Both paradigms exploit optimal substructure. The critical difference is the decision strategy:

	Greedy	Dynamic Programming
Decision at each step	Make the locally best choice; irrevocable	Evaluate <i>all</i> sub-results; choose globally best
Sub-results stored?	No — unchosen options discarded	Yes — table stores every sub-result
Why correct	Greedy-choice property holds globally	Principle of Optimality
Failure mode	Gets stuck locally when property fails (e.g., 0/1 Knapsack)	Always correct if recurrence is correct
Typical complexity	$O(n \log n)$ (sort once, scan)	$O(\text{states} \times \text{transition})$

Exchange argument (standard correctness proof technique): Assume an optimal solution differs from the greedy solution at some step. Show that *swapping* the optimal choice for the greedy choice yields a solution of equal or better value — contradicting the assumption that the original was strictly better.

2.2 Fractional Knapsack

Real-World Applications

- **Cargo loading:** A freight carrier maximises payload value by filling with the highest value-per-kg goods first; goods can be split (bags of grain, barrels of oil).
- **Budget allocation:** A project portfolio manager divides a fixed budget across initiatives ranked by return-on-investment; partial funding of a project is allowed.
- **Bandwidth scheduling:** Network slices are allocated proportionally to requests ordered by priority per unit bandwidth.

Problem Statement

Given n items, each with weight $w_i > 0$ and value $v_i > 0$, and a knapsack of capacity W , choose fractions $x_i \in [0, 1]$ to

$$\text{maximise } \sum_{i=1}^n v_i x_i \quad \text{subject to} \quad \sum_{i=1}^n w_i x_i \leq W.$$

Unlike the 0/1 variant, items may be taken fractionally.

Greedy Strategy

The value density $\rho_i = v_i/w_i$ measures value per unit weight. Sort items by ρ_i in decreasing order. Fill the knapsack greedily: take each item whole as long as capacity permits; take a fraction of the next item to fill the remaining space.

Why greediness is correct. If at any step we chose an item with lower density instead of the available higher-density item, we could swap the fractions to gain more value — contradicting optimality. The exchange argument formalises this: no rearrangement can do better once items are sorted by density.

Algorithm

Algorithm 4: FractionalKnapsack($w[1..n], v[1..n], W$)

```

1 Sort items so that  $v[1]/w[1] \geq v[2]/w[2] \geq \dots \geq v[n]/w[n]$ 
2  $total \leftarrow 0$ ;  $remaining \leftarrow W$ 
3 for  $i \leftarrow 1$  to  $n$  do
4   if  $remaining = 0$  then break
5    $take \leftarrow \min(w[i], remaining)$ 
6    $total \leftarrow total + (take/w[i]) \cdot v[i]$ 
7    $remaining \leftarrow remaining - take$ 
8 end
9 return  $total$ 

```

Trace Example 1

Items (w_i, v_i) : (3, 60), (2, 30), (5, 70), (4, 40). Capacity $W = 9$.

Value densities: $\rho_1 = 20$, $\rho_2 = 15$, $\rho_3 = 14$, $\rho_4 = 10$. Sorted order: item 1 ($\rho = 20$), item 2 ($\rho = 15$), item 3 ($\rho = 14$), item 4 ($\rho = 10$).

Step	Item	w_i	Take	Value added	Remaining cap
1	1 ($\rho = 20$)	3	3 (full)	60.0	6
2	2 ($\rho = 15$)	2	2 (full)	30.0	4
3	3 ($\rho = 14$)	5	4 (fraction 4/5)	56.0	0
Total value				146.0	

Item 3 is taken as 4/5 of its weight. Item 4 is never reached (capacity exhausted). Optimal value = 146.

Trace Example 2 (Levitin §9.3)

Items (w_i, v_i) : (1, 3), (4, 8), (3, 5), (5, 7). Capacity $W = 7$.

Value densities: $\rho_1 = 3.0$, $\rho_2 = 2.0$, $\rho_3 \approx 1.67$, $\rho_4 = 1.4$. Sorted: item 1, item 2, item 3, item 4.

Step	Item	w_i	Take	Value added	Remaining cap
1	1 ($\rho = 3.0$)	1	1 (full)	3.00	6
2	2 ($\rho = 2.0$)	4	4 (full)	8.00	2
3	3 ($\rho = 1.67$)	3	2 (fraction 2/3)	$3.\bar{3}$	0
Total value				$14.\bar{3}$	

Item 3 yields $2/3 \times 5 = 10/3 \approx 3.33$. Optimal = $14.\bar{3}$.

Complexity: Fractional Knapsack

Metric	Best	Average	Worst
Time	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$
Space	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$

The sort dominates: $\Theta(n \log n)$. The scan loop is $O(n)$. All cases equal because sorting always touches all n elements.

2.3 Prim's Algorithm (Minimum Spanning Tree)

Real-World Applications

- **Network design:** Laying cable, fibre-optic lines, or power lines between cities at minimum total cost; each pair of cities has an installation cost.
- **Cluster analysis:** Building a dendrogram for hierarchical clustering of data points; MST edges connect the closest pairs.
- **Approximation for TSP:** A 2-approximation for the metric TSP is built on the MST.
- **Computer networking:** Spanning-tree protocol (STP) in Ethernet uses MST-like logic to eliminate loops.

Definitions

A **spanning tree** of a connected undirected graph $G = (V, E)$ is a subgraph that is a tree (acyclic and connected) and includes all $|V|$ vertices. A **minimum spanning tree (MST)** is a spanning tree whose total edge weight is minimum.

Key property (cut property): For any cut of G , the minimum-weight crossing edge belongs to some MST. Prim's algorithm repeatedly applies this property to the cut $(V_T, V \setminus V_T)$ at each step.

Algorithm

Algorithm 5: Prim(G, s) — s is the starting vertex

```

1  $V_T \leftarrow \{s\}; \quad E_T \leftarrow \emptyset$ 
2 while  $|V_T| < |V|$  do
3   Find the minimum-weight edge  $(u, v)$  with  $u \in V_T, v \notin V_T$ 
4    $V_T \leftarrow V_T \cup \{v\}; \quad E_T \leftarrow E_T \cup \{(u, v)\}$ 
5 end
6 return  $(V_T, E_T)$                                      // MST
  
```

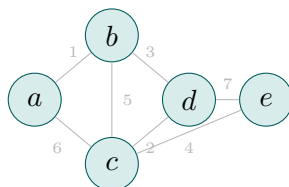
Implementation note: Using a min-priority queue keyed on the minimum edge weight to each vertex in $V \setminus V_T$ gives $O((|V| + |E|) \log |V|)$ time.

Trace Example 1

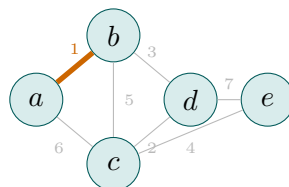
Graph G : vertices $\{a, b, c, d, e\}$, edges $ab:1, ac:6, bc:5, bd:3, cd:2, ce:4, de:7$. Start from a .

Step	V_T	Edge added	Min-weight crossing edge
0	$\{a\}$	—	$ab:1, ac:6$
1	$\{a, b\}$	$ab:1$	$ac:6, bc:5, bd:3$
2	$\{a, b, d\}$	$bd:3$	$ac:6, bc:5, cd:2, de:7$
3	$\{a, b, d, c\}$	$cd:2$	$ce:4, de:7$
4	$\{a, b, d, c, e\}$	$ce:4$	— (complete)

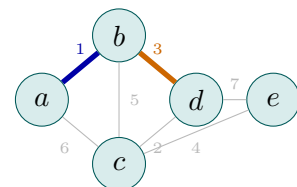
MST edges: $\{ab:1, bd:3, cd:2, ce:4\}$. Total weight = 10.



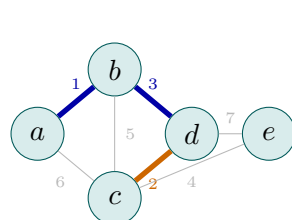
(a) Initial graph



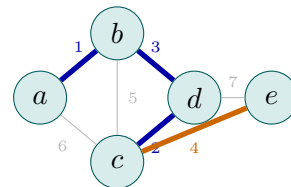
(b) Step 1: add $ab:1$



(c) Step 2: add $bd:3$



(d) Step 3: add $cd:2$



(e) Step 4: add $ce:4$ (MST complete)

Figure 2.1: Prim's Algorithm on 5-vertex graph. **Orange** = edge just added. **Blue** = MST edges so far. **Gray** = non-MST edges.

Trace Example 2 (Levitin §9.1)

Graph G : 6 vertices $\{a, b, c, d, e, f\}$. Edges: $ab:3$, $ac:6$, $bc:5$, $bd:2$, $cd:3$, $be:4$, $de:2$, $df:5$, $ef:4$. Start from a .

Step	V_T	Edge added	Candidates (min crossing)
0	$\{a\}$	—	$ab:3$, $ac:6$
1	$\{a, b\}$	$ab:3$	$ac:6$, $bc:5$, $bd:2$, $be:4$
2	$\{a, b, d\}$	$bd:2$	$ac:6$, $bc:5$, $cd:3$, $de:2$, $df:5$
3	$\{a, b, d, e\}$	$de:2$	$ac:6$, $bc:5$, $cd:3$, $df:5$, $ef:4$
4	$\{a, b, d, e, c\}$	$cd:3$	$df:5$, $ef:4$
5	$\{a, b, d, e, c, f\}$	$ef:4$	— (complete)

MST edges: $\{ab:3, bd:2, de:2, cd:3, ef:4\}$. Total weight = 14.

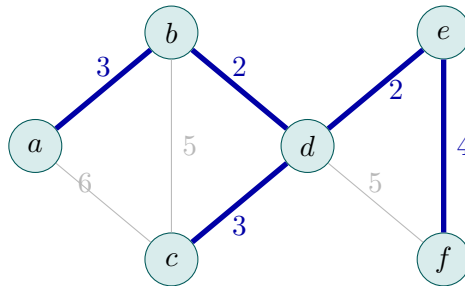


Figure 2.2: Final MST for 6-vertex graph (total weight = 14). **Blue** edges form the MST; gray edges are discarded.

Complexity: Prim's Algorithm

Implementation	Best	Average	Worst	Space
Adjacency matrix, linear scan	$\Theta(V^2)$	$\Theta(V^2)$	$\Theta(V^2)$	$\Theta(V^2)$
Binary heap + adjacency list	$\Theta((V + E) \log V)$	$\Theta((V + E) \log V)$	$\Theta((V + E) \log V)$	$\Theta(V)$
Fibonacci heap	$\Theta(E + V \log V)$	$\Theta(E + V \log V)$	$\Theta(E + V \log V)$	$\Theta(V)$

Derivation (binary heap): V EXTRACTMIN operations cost $O(V \log V)$; at most E DECREASEKEY operations cost $O(E \log V)$. Total: $O((V + E) \log V)$. On a dense graph ($E = \Theta(V^2)$), the matrix implementation's $\Theta(V^2)$ is better.

2.4 Kruskal's Algorithm (Minimum Spanning Tree)

Key Idea

Kruskal's algorithm is *edge-centric*: sort all edges by weight and add each edge to the MST if and only if it does not create a cycle. It relies on the **cycle property**: the maximum-weight edge in any cycle of G does not belong to any MST.

Union-Find Data Structure

The cycle check is maintained by the Union-Find (disjoint-set) structure. Each connected component of the growing forest is one set. Two operations:

- $\text{FIND}(u)$ — returns the representative (root) of u 's component.
- $\text{UNION}(u, v)$ — merges the components of u and v .

With path compression and union by rank, each operation costs $O(\alpha(n)) \approx O(1)$ amortised.

Algorithm

Algorithm 6: $\text{Kruskal}(G = (V, E))$

```

1 Sort  $E$  by weight:  $w(e_1) \leq w(e_2) \leq \dots \leq w(e_{|E|})$ 
2  $F \leftarrow \emptyset$  // forest of MST edges
3 foreach  $v \in V$  do
4   |  $\text{MakeSet}(v)$ 
5 end
6 foreach  $\text{edge } (u, v)$  in sorted order do
7   | if  $\text{FIND}(u) \neq \text{FIND}(v)$  then
8   |   |  $F \leftarrow F \cup \{(u, v)\}$ 
9   |   |  $\text{UNION}(u, v)$ 
10  | end
11 end
12 return  $F$ 

```

Trace Example 1 (same 5-vertex graph as Prim's)

Edges sorted by weight: $ab:1, cd:2, bd:3, ce:4, bc:5, ac:6, de:7$.

Step	Edge	Weight	Action	Components
0	—	—	Initialise	$\{a\}, \{b\}, \{c\}, \{d\}, \{e\}$
1	ab	1	Add (no cycle)	$\{a, b\}, \{c\}, \{d\}, \{e\}$
2	cd	2	Add (no cycle)	$\{a, b\}, \{c, d\}, \{e\}$
3	bd	3	Add (connects $\{a, b\}$ and $\{c, d\}$)	$\{a, b, c, d\}, \{e\}$
4	ce	4	Add (connects $\{a, b, c, d\}$ and $\{e\}$)	$\{a, b, c, d, e\}$ — done
5	bc	5	Skip (b, c same component)	—

$\text{MST} = \{ab:1, cd:2, bd:3, ce:4\}$. Weight = 10. Same MST as Prim's. ✓

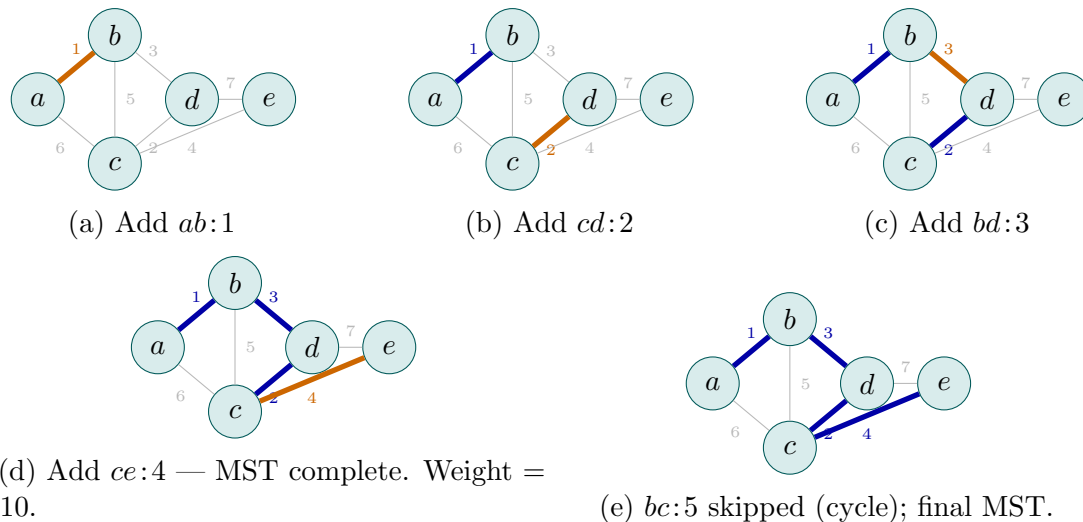


Figure 2.3: Kruskal's Algorithm: edge additions in weight order.

Trace Example 2 (same 6-vertex graph)Edges sorted: $bd:2$, $de:2$, $ab:3$, $cd:3$, $be:4$, $ef:4$, $df:5$, $bc:5$, $ac:6$.

Step	Edge	Wt	Result
1	bd	2	Add $\rightarrow \{b, d\}$
2	de	2	Add $\rightarrow \{b, d, e\}$
3	ab	3	Add $\rightarrow \{a, b, d, e\}$
4	cd	3	Add $\rightarrow \{a, b, c, d, e\}$
5	be	4	Skip (b and e in same component)
6	ef	4	Add $\rightarrow \{a, b, c, d, e, f\}$ — MST complete

MST = $\{bd:2, de:2, ab:3, cd:3, ef:4\}$. Weight = 14. Matches Prim's. ✓**Prim's vs. Kruskal's Comparison**

	Prim's	Kruskal's
Approach	Vertex-centric: grow one connected tree	Edge-centric: add cheapest safe edge anywhere
Data structure	Priority queue	Disjoint-set (Union-Find)
Time complexity	$O((V + E) \log V)$ with heap	$O(E \log E)$
Better for	Dense graphs ($E \approx V^2$)	Sparse graphs
Intermediate re- sult	Always a single connected tree	May be a forest of components

Complexity: Kruskal's Algorithm

Metric	Best	Average	Worst
Time	$\Theta(E \log E)$	$\Theta(E \log E)$	$\Theta(E \log E)$
Space	$\Theta(V)$	$\Theta(V)$	$\Theta(V)$

Derivation: Sorting E edges costs $\Theta(E \log E)$. Processing each edge with Union-Find costs $O(E \cdot \alpha(V)) \approx O(E)$, which is dominated by sorting. Since $\log E < \log V^2 = 2 \log V$, we can also write $O(E \log V)$.

2.5 Dijkstra's Algorithm — Single-Source Shortest Paths

Real-World Applications

- **GPS and map routing:** Google Maps, Apple Maps, and OpenStreetMap use Dijkstra-like algorithms to find the shortest (fastest, cheapest) driving route.
- **Network routing protocols:** OSPF (Open Shortest Path First) uses a Dijkstra-based computation to route IP packets.
- **Game AI:** Pathfinding for non-player characters on weighted tile maps.
- **Social networks:** Finding the shortest chain of acquaintances between two people.

Problem Statement

Given a weighted directed graph $G = (V, E, w)$ with $w(u, v) \geq 0$ for all edges (non-negative weights), and a source vertex s , find the shortest path from s to every other vertex.

Why non-negative weights? Dijkstra's greedy choice relies on the fact that once a vertex is settled (finalised), its tentative distance is already globally optimal. If negative-weight edges exist, a later path through a negative edge could improve a settled vertex's distance — breaking the greedy invariant. Use Bellman-Ford for graphs with negative weights.

Algorithm

Algorithm 7: Dijkstra(G, s)

```

1 foreach  $v \in V$  do
2   |  $d[v] \leftarrow \infty$ ;  $\pi[v] \leftarrow \text{nil}$ 
3 end
4  $d[s] \leftarrow 0$ ;  $Q \leftarrow V$  // min-priority queue keyed by  $d$ 
5 while  $Q \neq \emptyset$  do
6   |  $u \leftarrow \text{EXTRACTMIN}(Q)$ 
7   | foreach neighbour  $v$  of  $u$  do
8     | if  $d[u] + w(u, v) < d[v]$  then
9       | |  $d[v] \leftarrow d[u] + w(u, v)$ ;  $\pi[v] \leftarrow u$  // relax edge
10    | end
11  | end
12 end
13 return  $d$ 
  
```

Relaxation: The operation $d[v] \leftarrow \min(d[v], d[u] + w(u, v))$ is called relaxing edge (u, v) . After each vertex u is settled, its outgoing edges are relaxed.

Trace Example 1

Directed graph: $V = \{a, b, c, d\}$. Edges: $a \rightarrow b:1$, $a \rightarrow c:4$, $b \rightarrow c:2$, $b \rightarrow d:5$, $c \rightarrow d:1$.

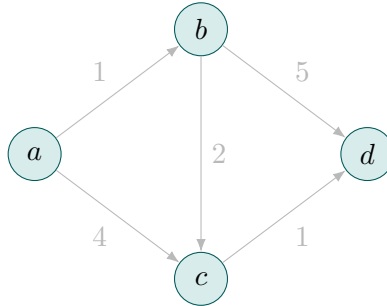


Figure 2.4: Input graph for Dijkstra's Example 1.

Step	Settle	Via	$d[a]$	$d[b]$	$d[c]$	$d[d]$
0	—	—	0	∞	∞	∞
1	a	—	0	$\min(\infty, 0+1) = \mathbf{1}$	$\min(\infty, 0+4) = \mathbf{4}$	∞
2	b	a	0	1	$\min(4, 1+2) = \mathbf{3}$	$\min(\infty, 1+5) = \mathbf{6}$
3	c	b	0	1	3	$\min(6, 3+1) = \mathbf{4}$
4	d	c	0	1	3	4

Final shortest distances from a : $d[a] = 0$, $d[b] = 1$, $d[c] = 3$, $d[d] = 4$. Shortest paths: $a \rightarrow b$ (cost 1), $a \rightarrow b \rightarrow c$ (cost 3), $a \rightarrow b \rightarrow c \rightarrow d$ (cost 4).

Trace Example 2 (Levitin §9.3)

Directed graph: $V = \{a, b, c, d, e\}$. Edges: $a \rightarrow b:4$, $a \rightarrow c:2$, $b \rightarrow c:1$, $b \rightarrow d:5$, $c \rightarrow b:1$, $c \rightarrow d:8$, $c \rightarrow e:10$, $d \rightarrow e:2$.

Step	Settle	Via	$d[a]$	$d[b]$	$d[c]$	$d[d]$	$d[e]$
0	—	—	0	∞	∞	∞	∞
1	a	—	0	4	2	∞	∞
2	c	a	0	$\min(4, 3) = \mathbf{3}$	2	$\min(\infty, 10) = \mathbf{10}$	$\min(\infty, 12) = \mathbf{12}$
3	b	c	0	3	2	$\min(10, 8) = \mathbf{8}$	12
4	d	b	0	3	2	8	$\min(12, 10) = \mathbf{10}$
5	e	d	0	3	2	8	10

Final: $d[a] = 0$, $d[b] = 3$, $d[c] = 2$, $d[d] = 8$, $d[e] = 10$. Shortest path to e : $a \rightarrow c \rightarrow b \rightarrow d \rightarrow e$ (cost 10).

Complexity: Dijkstra's Algorithm

Implementation	Best	Average	Worst	Space
Adjacency matrix, linear scan	$\Theta(V^2)$	$\Theta(V^2)$	$\Theta(V^2)$	$\Theta(V^2)$
Binary heap + adj. list	$\Theta((V + E) \log V)$	$\Theta((V + E) \log V)$	$\Theta((V + E) \log V)$	$\Theta(V)$

Derivation (binary heap): V EXTRACTMIN + up to E DECREASEKEY operations, each $O(\log V)$ with a binary heap: $T = O(V \log V + E \log V) = O((V + E) \log V)$. No early termination: all cases equal (the loop always processes every vertex).

2.6 Huffman Coding

Real-World Applications

- **File compression:** The DEFLATE algorithm (used in ZIP, gzip, PNG) combines Huffman coding with LZ77.
- **Image and audio compression:** JPEG uses Huffman coding for the entropy-coding stage; MP3 uses a modified Huffman scheme.
- **Data transmission:** Huffman codes minimise the expected number of bits per symbol sent over a channel, given known symbol frequencies.
- **Compiler design:** Token encoding in intermediate representations.

Problem Statement

Given an alphabet $\Sigma = \{c_1, \dots, c_n\}$ with known character frequencies $f_1, \dots, f_n$, construct a **prefix-free** (no code is a prefix of another) binary code that minimises the total number of bits needed to encode a message.

The **weighted path length (WPL)** of a code tree is:

$$\text{WPL} = \sum_{i=1}^n f_i \cdot \ell_i$$

where ℓ_i is the depth (code length) of character c_i in the binary tree. An optimal prefix-free code minimises WPL.

Greedy Strategy: Build the Huffman Tree Bottom-Up

1. Create a leaf node for each character; insert all leaves into a min-heap keyed by frequency.
2. While the heap has more than one node:
 - (a) Extract the two minimum-frequency nodes x and y .
 - (b) Create an internal node z with $\text{freq}(z) = \text{freq}(x) + \text{freq}(y)$; $z.\text{left} = x$, $z.\text{right} = y$.
 - (c) Insert z back into the heap.
3. The remaining node is the root of the Huffman tree.

The greedy choice — always merge the two least frequent symbols — is optimal by the *sibling property*: any optimal tree can be transformed into one where the two least-frequent symbols are siblings at the deepest level.

Algorithm

Algorithm 8: Huffman($C[1..n]$) — each entry has `.char` and `.freq`

```

1 Build min-heap  $Q$  from  $C$  keyed on .freq
2 for  $i \leftarrow 1$  to  $n - 1$  do
3    $x \leftarrow \text{EXTRACTMIN}(Q)$ 
4    $y \leftarrow \text{EXTRACTMIN}(Q)$ 
5   Create new node  $z$ :  $z.\text{left} \leftarrow x$ ,
6    $z.\text{right} \leftarrow y$ ,
7    $z.\text{freq} \leftarrow x.\text{freq} + y.\text{freq}$ 
8    $\text{INSERT}(Q, z)$ 
9 end
10 return  $\text{EXTRACTMIN}(Q)$                                 // root of Huffman tree
```

Trace Example 1 (Levitin §9.4)

Characters and frequencies: A:35, B:10, C:20, D:20, E:15.

Step-by-step merge sequence:

Step	Heap contents (sorted by freq)	Action
0	B:10, E:15, C:20, D:20, A:35	Initial heap
1	C:20, D:20, [BE]:25, A:35	Extract B:10, E:15 $\rightarrow$ create [BE]:25
2	[BE]:25, A:35, [CD]:40	Extract C:20, D:20 $\rightarrow$ create [CD]:40
3	[CD]:40, [ABE]:60	Extract [BE]:25, A:35 $\rightarrow$ create [ABE]:60
4	root:100	Extract [CD]:40, [ABE]:60 $\rightarrow$ create root:100

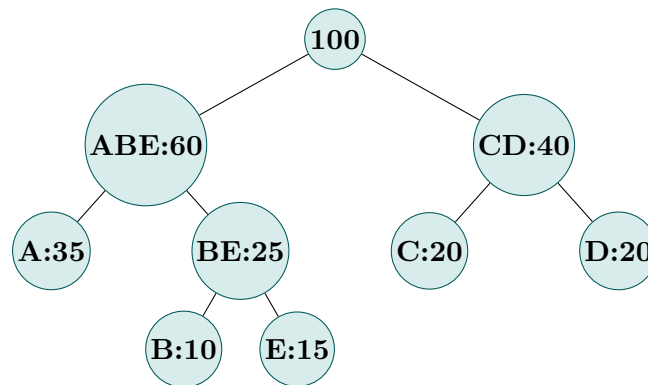


Figure 2.5: Huffman tree for A:35, B:10, C:20, D:20, E:15. Left branch = 0, Right branch = 1.

Codes and WPL:

Char	Freq	Code	Length ℓ	Contribution $f \cdot \ell$
A	35	00	2	70
B	10	010	3	30
E	15	011	3	45
C	20	10	2	40
D	20	11	2	40
WPL				225

Fixed-length encoding would need 3 bits per symbol (5 symbols $\Rightarrow \lceil \log_2 5 \rceil = 3$), giving $100 \times 3 = 300$ bits. Huffman reduces this to 225 bits — a **25% saving**.

Trace Example 2

Characters and frequencies: a:1, b:4, c:5, d:3, e:2.

Step	Heap contents	Action
0	a:1, e:2, d:3, b:4, c:5	Initial
1	d:3, [ae]:3, b:4, c:5	Extract a:1, e:2 $\rightarrow$ [ae]:3
2	b:4, c:5, [dae]:6	Extract d:3, [ae]:3 $\rightarrow$ [dae]:6
3	[dae]:6, [bc]:9	Extract b:4, c:5 $\rightarrow$ [bc]:9
4	root:15	Extract [dae]:6, [bc]:9 $\rightarrow$ root:15

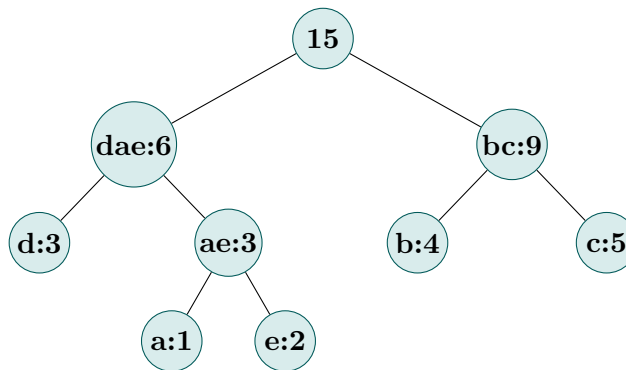


Figure 2.6: Huffman tree for a:1, b:4, c:5, d:3, e:2.

Codes: d=00, a=010, e=011, b=10, c=11. WPL = $3 \times 2 + 1 \times 3 + 2 \times 3 + 4 \times 2 + 5 \times 2 = 6 + 3 + 6 + 8 + 10 = 33$. Fixed 3-bit: $15 \times 3 = 45$. Saving: 12 bits (27%).

Complexity: Huffman Coding

Metric	Best	Average	Worst
Time	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$
Space	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$

Derivation: Building the heap costs $\Theta(n)$. The loop runs $n - 1$ times; each iteration performs 2 EXTRACTMIN and 1 INSERT, each $O(\log n)$. Total: $\Theta(n \log n)$. The tree itself has $2n - 1$ nodes, giving $\Theta(n)$ space.

2.7 End-of-Chapter Questions

Numerical Questions (Levitin-style)

1. Apply FractionalKnapsack to items (w, v) : $(2, 5), (3, 4), (4, 3), (5, 2)$, $W = 8$. Show sorted order, amounts taken, and total value.
2. Trace Prim's algorithm on a 4-vertex graph K_4 with edge weights 12, 8, 11, 7, 9, 10 for edges 1-2, 1-3, 1-4, 2-3, 2-4, 3-4. Starting from vertex 1, record the edge added at each step.
3. Apply Kruskal's to the same K_4 and show all Union-Find states. Verify you get the same MST as Prim's.
4. Run Dijkstra's from s on the graph with edges $s \rightarrow a:7$, $s \rightarrow b:2$, $b \rightarrow a:3$, $b \rightarrow$

- $c:4, a \rightarrow c:1$. Show the relaxation table. What is the shortest path from s to c ?
5. Run Dijkstra's from s on a graph with a negative edge weight. Show where the algorithm gives an incorrect answer.
 6. Build a Huffman tree for frequencies: $a:5, b:2, c:1, d:3, e:4$. Compute the WPL and compare with fixed-width encoding.
 7. For Huffman with $n = 1$ symbol, what is the code length? What special case must the algorithm handle?
 8. Prove that swapping two items of different densities in FractionalKnapsack cannot improve the total value (exchange argument).
 9. Explain why Dijkstra's algorithm is a greedy algorithm. What is the greedy choice at each step? What is the greedy-choice property?
 10. For a graph with $V = 1000$ and $E = 5000$, compare the running time of Prim's (binary heap) and Kruskal's. Which is better?

LeetCode Practice

1. 1584 — Min Cost to Connect All Points (Prim's or Kruskal's on a complete graph)
2. 1135 — Connecting Cities With Minimum Cost (Kruskal's)
3. 743 — Network Delay Time (Dijkstra's)
4. 1091 — Shortest Path in Binary Matrix (BFS / Dijkstra)
5. 1046 — Last Stone Weight (min-heap greedy)
6. 1167 — Minimum Cost to Connect Sticks (Huffman-style)
7. 435 — Non-overlapping Intervals (greedy interval scheduling)
8. 630 — Course Schedule III (greedy + heap)

Chapter 3

Backtracking

Chapter Outcomes

You will construct state-space trees for constraint-satisfaction problems, implement the *promising* function to prune infeasible and suboptimal branches, trace backtracking for the n -Queens and Subset-Sum problems, and analyse the worst-case exponential complexity of backtracking.

3.1 Strategy Overview

Backtracking is a systematic depth-first search through a *state-space tree* that represents all partial candidates for a solution. At each node, a **promising function** determines whether the partial candidate can still lead to a valid solution. If it cannot (*not promising*), the entire subtree rooted at that node is abandoned — the algorithm *backtracks* to the parent and tries the next branch.

Key Concepts

- **State-space tree:** A rooted tree where the root represents the empty partial solution; each level i represents the i -th decision; each node is a partial assignment. Leaf nodes are either complete solutions or dead ends.
- **Promising function:** Returns **true** if a node *might* extend to a valid solution; **false** if it can be eliminated without loss.
- **Pruning:** Cutting off an unpromising subtree. Effective pruning dramatically reduces the number of nodes explored compared to exhaustive enumeration ($n!$ or 2^n).

General Template

Algorithm 9: Backtrack(*node*)

```

1 if node is a solution then
2   | Output node; return
3 end
4 if promising(node) then
5   | foreach child c of node do
6     | Backtrack(c)
7   | end
8 end

```

Backtracking vs. Brute Force

	Brute Force	Backtracking
Search	Exhaustive: tries every candidate	DFS with pruning: abandons branches early
State-space	Fully explored	Pruned
Efficiency	$O(n!)$ or $O(2^n)$ in all cases	Often much better in practice
Completeness	Finds all solutions	Finds all solutions (pruning only removes dead ends)

3.2 n -Queens Problem

Real-World Applications

- **Constraint satisfaction problems (CSP):** The n -Queens pattern underlies general CSP solvers used in scheduling, configuration, and planning.
- **VLSI design:** Placement of non-interfering components on a chip.
- **Parallel computing:** Non-conflicting task assignment across processing units.
- **Combinatorics research:** Counting and enumerating the solutions forms the basis of many combinatorial studies.

Problem Statement

Place n queens on an $n \times n$ chessboard such that no two queens attack each other — no two queens share the same row, column, or diagonal.

State-Space Representation

- One queen per row (guaranteed by construction).
- State at level i : a vector $(c_1, c_2, \dots, c_i)$ where c_k is the column of the queen in row k .
- Branching factor at level i : up to n column choices.

- Total nodes without pruning: $1 + n + n^2 + \dots + n^n = O(n^n)$.

Promising Function

Queen at (i, c_i) is *promising* if it does not conflict with any already-placed queen (j, c_j) for $j < i$:

1. **Column:** $c_i \neq c_j$
2. **Diagonal:** $|c_i - c_j| \neq |i - j|$

Algorithm

Algorithm 10: Queens($i, col[1..n]$) — call as Queens(1, col)

```

1 if  $i > n$  then
2   | Output  $col$ ; return                                // complete solution
3 end
4 for  $c \leftarrow 1$  to  $n$  do
5   |  $col[i] \leftarrow c$ 
6   | if Promising( $i, col$ ) then
7   |   | Queens( $i + 1, col$ )
8   | end
9 end

```

Algorithm 11: Promising($i, col[1..n]$)

```

1 for  $j \leftarrow 1$  to  $i - 1$  do
2   | if  $col[i] = col[j]$  or  $|col[i] - col[j]| = |i - j|$  then return false
3 end
4 return true

```

Trace Example 1: 4-Queens

We trace the DFS through the state-space tree for $n = 4$. Each node is labelled $(c_1, \dots, c_k)$ for the column choices made so far. Nodes marked $\times$ fail the promising function.

Row 1 — try $c_1 = 1$:

- $c_2 = 1$: same column as $(1, 1) \rightarrow$ **fail**.
- $c_2 = 2$: diagonal $|1 - 2| = |1 - 2| = 1 \rightarrow$ **fail**.
- $c_2 = 3$: OK. Now $c_3 = 1$: same col as row 1 $\rightarrow$ fail. $c_3 = 2$: diagonal with $(2, 3) \rightarrow$ fail. $c_3 = 3$: same col as row 2 $\rightarrow$ fail. $c_3 = 4$: diagonal with $(2, 3)$ ($|2 - 3| = |3 - 4| = 1$) $\rightarrow$ **fail**. All of row 3 fails, backtrack.
- $c_2 = 4$: OK. $c_3 = 2$: check vs $(1, 1)$: OK; vs $(2, 4)$: diagonal $|2 - 3| = |4 - 2|$? $1 \neq 2 \rightarrow$ OK. Now c_4 : col 1 same as row 1 $\rightarrow$ fail; col 2 same as row 3 $\rightarrow$ fail; col 3: vs $(3, 2)$: diagonal $|3 - 4| = |2 - 3| = 1 \rightarrow$ fail; col 4: same col as row 2 $\rightarrow$ fail. All fail, backtrack.

$c_1 = 1$ yields **no solution**.

Row 1 — try $c_1 = 2$:

- $c_2 = 1$: diagonal with $(1, 2)$: $|1-2| = |2-1| = 1 \rightarrow$ fail.
- $c_2 = 3$: diagonal $|1-2| = |2-3| = 1 \rightarrow$ fail.
- $c_2 = 4$: OK.
 - $c_3 = 1$: vs $(1, 2)$: $|2-1| \neq |1-3|$ OK; vs $(2, 4)$: $|2-4| \neq |3-1|$ ($2 \neq 2$?) Actually $|2-4| = 2$, $|3-1| = 2 \rightarrow$ diagonal $\rightarrow$ fail. Wait: $|c_3 - c_2| = |1-4| = 3$, $|3-2| = 1$. $3 \neq 1 \rightarrow$ OK. Re-check vs $(1, 2)$: $|c_3 - c_1| = |1-2| = 1$, $|3-1| = 2$. $1 \neq 2 \rightarrow$ OK. Place $(3, 1)$.
 - * $c_4 = 3$: vs $(1, 2)$: $|3-2| = 1$, $|4-1| = 3 \rightarrow$ OK. vs $(2, 4)$: $|3-4| = 1$, $|4-2| = 2 \rightarrow$ OK. vs $(3, 1)$: $|3-1| = 2$, $|4-3| = 1 \rightarrow$ OK. **SOLUTION:** $(2, 4, 1, 3) \checkmark$

Row 1 — try $c_1 = 3$:

- $c_2 = 1$: OK.
 - $c_3 = 4$: vs $(1, 3)$: $|4-3| = 1$, $|3-1| = 2 \rightarrow$ OK. vs $(2, 1)$: $|4-1| = 3$, $|3-2| = 1 \rightarrow$ OK. Place $(3, 4)$.
 - * $c_4 = 2$: vs $(1, 3)$: $|2-3| = 1$, $|4-1| = 3 \rightarrow$ OK. vs $(2, 1)$: $|2-1| = 1$, $|4-2| = 2 \rightarrow$ OK. vs $(3, 4)$: $|2-4| = 2$, $|4-3| = 1 \rightarrow$ OK. **SOLUTION:** $(3, 1, 4, 2) \checkmark$

Row 1 — $c_1 = 4$: Symmetric to $c_1 = 1$ by the board's left-right symmetry; yields no new solutions.

Summary of solutions found:

1. $(c_1, c_2, c_3, c_4) = (2, 4, 1, 3)$
2. $(c_1, c_2, c_3, c_4) = (3, 1, 4, 2)$

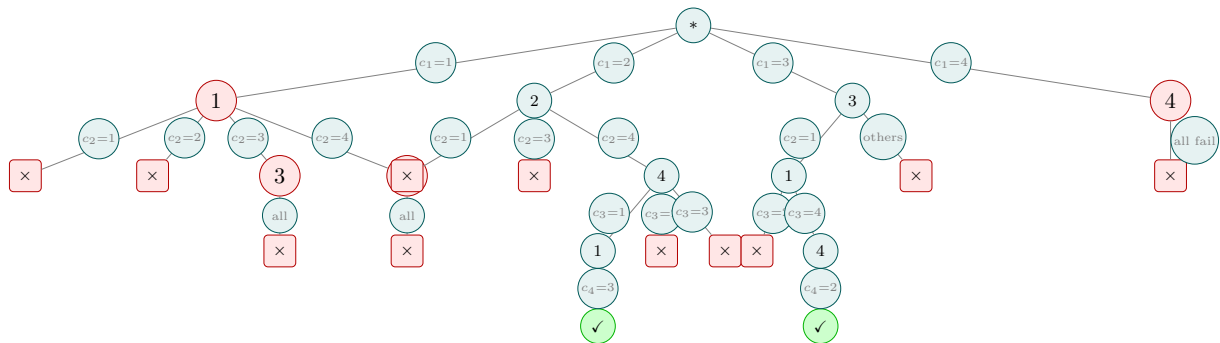


Figure 3.1: State-space tree for 4-Queens. **Teal nodes** = promising partial solutions being explored. **Red nodes** / $\times$ = pruned (not promising). $\checkmark$ = complete solutions. Node labels show the column c_i at the level- i decision.

Solution Board: $(2, 4, 1, 3)$

	Q		
			Q
Q			
		Q	

		Q	
Q			
			Q
	Q		

Solution 1: (2, 4, 1, 3)

Solution 2: (3, 1, 4, 2)

Complexity: n -Queens

Metric	Best	Average	Worst
Time	$\Omega(n)$	problem-dependent	$O(n!)$
Space	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$

Without pruning: The state-space tree has $O(n^n)$ nodes. **With pruning:** Experimentally fewer than $n!$ nodes are visited. For $n = 8$: 92 solutions found out of $8! = 40320$ naive candidates; backtracking explores only ≈ 15720 nodes. Space is $\Theta(n)$ — only the current path is stored on the recursion stack.

3.3 Subset-Sum Problem

Real-World Applications

- **Cryptography:** The knapsack cryptosystem (Merkle-Hellman) is based on subset-sum hardness; the trapdoor uses a superincreasing sequence.
- **Resource allocation:** Can a set of tasks exactly fill a time slot?
- **Partition problem:** Can a set of numbers be split into two equal-sum groups? (Directly reducible to subset-sum with $d = \text{sum}/2$.)
- **Change-making:** Can exact change be made from a set of coin denominations?

Problem Statement

Given a set of positive integers $S = \{w_1, w_2, \dots, w_n\}$ (sorted in increasing order) and a target d , find all subsets of S that sum exactly to d .

State-Space Representation

Each path from root to a leaf in the binary state-space tree represents a subset: at level i , the left child includes w_i and the right child excludes it. The tree has 2^n leaves — backtracking prunes it using two conditions.

Promising Function

At a node with current sum sum at level i (items $w_1, \dots, w_i$ decided):

$$\text{remSum} = \sum_{j=i+1}^n w_j$$

The node is **promising** if:

1. $\text{sum} + \text{remSum} \geq d$ (upper bound: can still reach d)
2. $\text{sum} + w_{i+1} \leq d$ (lower bound: including the next item doesn't overshoot)

Condition 1 fails if the total remaining sum can never reach d (prune right subtree). Condition 2 fails if even including the next item would exceed d (prune left subtree).

Algorithm

Algorithm 12: SubsetSum($i, \text{sum}, \text{remSum}, \text{include}[1..n], w[1..n], d$)

```

1 if  $\text{sum} = d$  then
2   | Output  $\{w_k : \text{include}[k] = \text{true}\}$ ; return
3 end
4 if  $i > n$  then
5   | return
6 end
7 if  $\text{sum} + \text{remSum} \geq d$  and  $\text{sum} + w[i] \leq d$  then
8   |  $\text{include}[i] \leftarrow \text{true}$ 
9   | SubsetSum( $i + 1, \text{sum} + w[i], \text{remSum} - w[i], \text{include}, w, d$ )
10  |  $\text{include}[i] \leftarrow \text{false}$ 
11 end
12 if  $\text{sum} + \text{remSum} - w[i] \geq d$  and  $\text{sum} \leq d$  then
13   | SubsetSum( $i + 1, \text{sum}, \text{remSum} - w[i], \text{include}, w, d$ )
14 end

```

How the Algorithm Works

The array $\text{include}[1..n]$ is *shared* across the whole recursion. It records the decisions made along the **current** root-to-node path: $\text{include}[k] = \text{true}$ means “ w_k is in the subset being built right now.” When $\text{sum} = d$ (line 1) the output step reads this array to print exactly which weights form the solution.

The left branch — “make, explore, undo”. The block $\text{include}[i] \leftarrow \text{true}$, recurse, $\text{include}[i] \leftarrow \text{false}$ is the standard backtracking idiom for the $\text{include } w_i$ subtree:

1. $\text{include}[i] \leftarrow \text{true}$ (**make the choice**). We are entering the left child, where w_i is taken. The recursive call also reflects the choice in its *values*: $\text{sum} + w[i]$ (running sum grows) and $\text{remSum} - w[i]$ (w_i is no longer undecided).
2. **recurse (explore)**. Visit *every* subset that contains w_i .
3. $\text{include}[i] \leftarrow \text{false}$ (**undo the choice**). The left subtree is finished, so we must clear w_i ’s flag *before* moving on to the right branch and before returning to the parent. Because $\text{include}[]$ is a single shared array, a leftover **true** would leak into those sibling/ancestor explorations and make them report w_i as part of subsets that do not contain it.

So true means “included *for the duration of this subtree only*”; the reset restores a clean state for the alternative.

The right branch — exclude w_i . The second **if** is the *exclude* w_i child: the recursive call keeps sum unchanged (we add nothing) but still passes $\text{remSum} - w[i]$, since w_i is now decided (rejected) and no longer available. No assignment to $\text{include}[i]$ is needed — it is already **false** from the undo step above. Its two-part guard is the bounding test for this subtree:

- $sum + remSum - w[i] \geq d$: after discarding w_i , the largest sum still reachable is the current sum plus *all remaining weights except w_i* . If even that maximum falls below d , this branch can never reach d , so the right subtree is pruned. (This is Condition 1 of the promising function, applied after removing w_i .)
- $sum \leq d$: a lower-bound sanity check that we have not already overshoot d . It always holds here (weights are positive and nothing is added) and is kept for symmetry with the left branch's guard.

Trace Example 1 (Levitin §12.1)

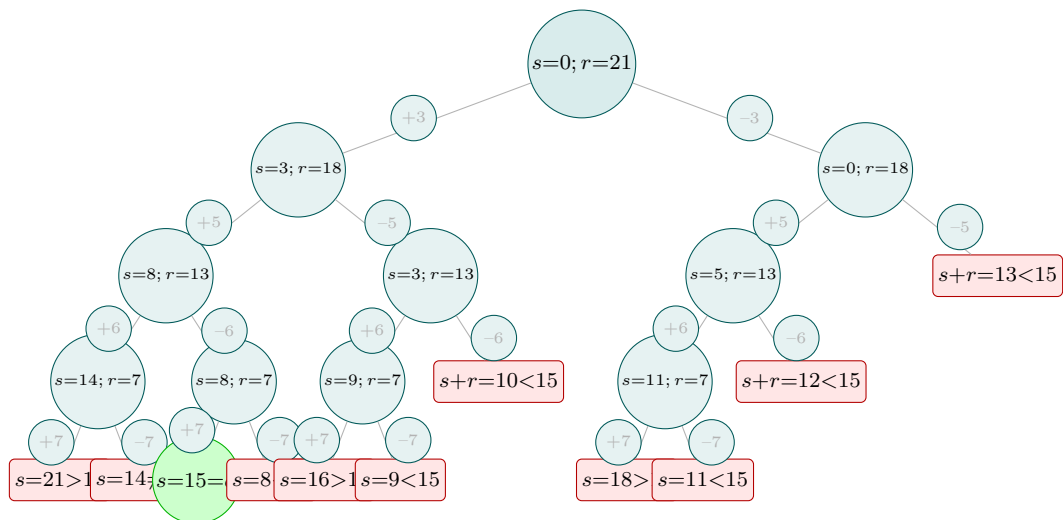
$$S = \{3, 5, 6, 7\}, d = 15. \quad \text{Initial: sum} = 0, \text{remSum} = 21.$$


Figure 3.2: Subset-Sum state-space tree for $S = \{3, 5, 6, 7\}$, $d = 15$. Each node shows current sum s and remaining sum r . **Green** = solution found. **Pruned nodes** = cut by promising function. Branch labels show element added (+) or skipped (-).

Solution found: $\{3, 5, 7\}$ with $3 + 5 + 7 = 15 = d$. ✓

Where does a pruned right leg “exit” in the algorithm? A right leg is the *exclude* w_i branch. It is cut by the guard of the **second if** — the line $sum + remSum - w[i] \geq d$ **and** $sum \leq d$. When the first conjunct is false the body (the right recursive call) is skipped, so the child is *never entered*: the rejection is made by the *parent*, not by a line inside the child. This is why the procedure body seems to test only sum (lines $sum = d$ and $i > n$): the $remSum$ bound is not a separate **return**, it lives entirely inside the two **if** guards.

The quantity actually tested for an exclude branch is the *upper bound on what is still reachable after dropping w_i* , namely $UB = sum + (remSum - w[i])$:

$$\text{UB} = \text{sum} + (\text{remSum} - w[i]) < d \implies \text{prune the right subtree.}$$

Each pruned right leg in the tree is labelled with this bound. For the exclude-6 child of $s=3, r=13$ deciding $w_i=6$, the child has $s=3, r=7$, so $s+r = 10 < 15$ and it is dropped (the guard evaluates $3 + 13 - 6 = 10 \not\geq 15$). When w_i is the last item, $remSum - w[i] = 0$

and the bound collapses to $sum < d$ (e.g. $s=8<15$) — it is the same condition, just with no remaining term left.

Trace Example 2

$S = \{2, 4, 5, 7\}$, $d = 9$. Initial: $sum = 0$, $remSum = 18$.

Path	sum	remSum	UB = $s + r$	Action
Include 2	2	16	18	Promising
Include 4	6	12	18	Promising
Include 5	11	7	18	$s = 11 > 9$ — prune
Exclude 5: in- clude 7	13	0	13	$s = 13 > 9$ — prune
Exclude 5: excl 7	6	0	6	$s \neq 9$, $r = 0$ — dead end
Exclude 4	2	12	14	Promising
Include 5	7	7	14	Promising
Include 7	14	0	14	$s = 14 > 9$ — prune
Exclude 7	7	0	7	$s \neq 9$ — dead end
Exclude 5: in- clude 7	9	0	9	$s = 9 = d$ SOLU- TION $\{2, 7\}$ ✓
Exclude 2	0	16	16	Promising
Include 4	4	12	16	Promising
Include 5	9	7	16	$s = 9 = d$ SOLU- TION $\{4, 5\}$ ✓
Exclude 5: in- clude 7	11	0	11	$s = 11 > 9$ — prune
Exclude 4: include 5	5	7	12	Promising
Include 7	12	0	12	$s = 12 > 9$ — prune
Exclude 4: excl 5	0	7	7	UB = $7 < 9$ — prune

Solutions found: $\{2, 7\}$ (sum = 9) and $\{4, 5\}$ (sum = 9). ✓

Complexity: Subset-Sum Backtracking

Metric	Best	Average	Worst
Time	$\Omega(1)$	Problem-dependent	$O(2^n)$
Space	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$

Without pruning: the binary state-space tree has $2^{n+1} - 1$ nodes — exponential.

With pruning: significantly fewer nodes are expanded in practice. The space $\Theta(n)$ stores only the current path (recursion stack depth n).

3.4 End-of-Chapter Questions

Numerical Questions

- Trace the 4-Queens algorithm starting with $c_1 = 4$. Verify no solutions are found (symmetric to $c_1 = 1$).
- List all 92 solutions to the 8-Queens problem without computing them — explain why this number is found by backtracking efficiently.
- Apply the Subset-Sum algorithm to $S = \{1, 3, 4, 5\}$, $d = 7$. Draw the state-space tree and identify all solutions.
- Apply Subset-Sum to $S = \{3, 4, 5, 6\}$, $d = 9$. Compare the number of nodes explored vs. the brute-force $2^4 = 16$ subsets.
- Modify the promising function for Subset-Sum to handle *multisets* (repeated elements).
- Why is the backtracking state-space tree for n -Queens of depth n and not n^2 ?
- For the n -Queens problem, show that the number of nodes at level i is at most $n!/(n-i)!$ in the worst case.
- Design a backtracking algorithm for the **graph colouring** problem: colour the vertices of a graph with at most k colours such that no two adjacent vertices share a colour. What is the promising function?

LeetCode Practice

- 51 — N-Queens
- 52 — N-Queens II (count only)
- 39 — Combination Sum (subset-sum variant)
- 40 — Combination Sum II
- 416 — Partition Equal Subset Sum (DP variant of subset-sum)
- 78 — Subsets
- 90 — Subsets II (with duplicates)
- 47 — Permutations II (backtracking with pruning)

Chapter 4

Branch and Bound

Chapter Outcomes

You will distinguish Branch and Bound from backtracking by the role of bounding functions, construct B&B state-space trees for the 0/1 Knapsack and TSP problems, compute upper bounds using fractional relaxation and lower bounds using minimum-edge arguments, and explain best-first branch-and-bound.

4.1 Strategy Overview

Branch and Bound (B&B) is backtracking applied to optimisation problems, with *one* addition: at every node we compute a **bound** — an optimistic estimate of the best value reachable below that node — and compare it against the best complete solution found so far. If the bound cannot beat the best-so-far, the whole subtree is discarded. That single idea is all B&B adds; the state-space tree, branching, and constraint checks are inherited unchanged from backtracking. The differences are confined to the optimisation goal and the bound:

	Backtracking	Branch and Bound
Goal	<i>Feasibility</i> / enumeration	<i>Optimisation</i> : find best solution
Pruning tool	Feasibility / constraint check	Bounding function : prune when $\text{bound} \leq \text{best known}$
Search order	Depth-first (implicit)	Best-first (most promising node first)
Bound type	None (only feasibility)	Computed upper bound (maximisation) or lower bound (minimisation)

What B&B Adds: the Bound

Everything except the bound is ordinary backtracking. Concretely:

- **Branching** — identical to backtracking: split a node into children by fixing one decision (include/exclude an item, choose the next city).
- **Bounding** — the one new step: an *optimistic* estimate of the best value any completion below this node could reach. Maximisation gives an upper bound ($UB \geq$ every solution in the subtree); minimisation a lower bound ($LB \leq$ every solution). Being optimistic is what makes pruning safe: a node is dropped only when even its best case cannot win, so the optimum is never discarded.
- **Pruning / termination** — a node is dropped for any of three reasons: (1) its bound is no better than the best solution so far; (2) it is infeasible, a constraint violated (the *same* test as backtracking); or (3) it is a single complete solution — just compare it with the best and update.

Reason (2) is pure backtracking; reasons (1) and (3) are the bound at work. The order in which leaves are expanded is a free choice: **best-first** (branch next from the leaf with the best bound, using a priority queue) reaches a good solution early, which sharpens the best-so-far and prunes more by reason (1). It affects efficiency, not correctness.

Intuition: the traffic-jam analogy

You are driving across a city to a fixed destination during a traffic jam.

- **Backtracking** only asks: *does this road eventually reach the destination?* Any route that arrives is *feasible*; roads that dead-end or are blocked are pruned. It keeps *every* route that gets you there — good or bad — because its only test is feasibility.
- **Branch and Bound** adds a metric — say, the **number of traffic signals** — and seeks the route with the *fewest*. As you drive a partial route, you estimate *optimistically*:

(signals so far) + (fewest signals the rest could possibly have).

If even that best case is no better than the best complete route already found ($\geq$ *best*), this route *cannot* win — abandon it. Each time you complete a route, update *best*.

The priority queue simply tells you which promising route to drive next — the one looking shortest so far — so you reach a good complete route early and can eliminate the rest sooner.

Template — side by side with backtracking

Written in the *same recursive skeleton*, the two are line-for-line comparable. Both keep the *same* promising (feasibility) test — B&B does feasibility too. B&B adds exactly **one** line, the **bound** (Exit 1, marked NEW); the placeholder comment in BACKTRACK marks where that line is absent. Read across.

Algorithm 13: Backtrack(*node*)

```

1 if not promising(node) then return
    // Exit 2: infeasible
/* no bound test --- backtracking
   has no Exit 1 */
2 if node is a solution then output;
   return
    // Exit 3: complete
3 foreach child c of node do
4   | BACKTRACK(c) // branch
5 end

```

Algorithm 14: BranchBound(*node*)

```

1 if not promising(node) then return
    // Exit 2: infeasible
2 if bound(node) ≤ best then return
    // Exit 1: NEW if node is a solution
    then best ← max(best, val(node));
    return
    // Exit 3
3 foreach child c of node do
4   | BRANCHBOUND(c) // branch
5 end

```

Driver for B&B: $best \leftarrow -\infty$; call BRANCHBOUND(*root*); return *best*. Recursion here is depth-first; replacing it with a priority-queue driver that expands the most promising live node first gives **best-first** B&B — a change of *order* only, isolating the bound as the one conceptual addition.

promising feasibility test — present in *both*; B&B inherits it unchanged.
Branch generate the children of a node (FOREACH CHILD) — identical in both.
Exit 2 not promising: a constraint is violated. *Same line in both*.
Exit 1 $bound(node) \leq best$: even optimistically this node cannot beat the best solution so far. *The one line B&B adds*.
Exit 3 a complete solution: backtracking outputs it; B&B compares it with *best* and updates.

4.2 From Feasibility to Optimality: Why These Problems Need a Bound

Every backtracking example so far asked only a **feasibility** (yes/no) question — there was nothing to optimise, so there was no bound to compute:

Problem	Feasibility question (the only one)	Optimality question
<i>n</i> -Queens	Can all <i>n</i> queens be placed with none attacking another?	<i>none</i> — any valid board is equally good
Subset-Sum	Does some subset add up exactly to the target <i>d</i> ?	<i>none</i> — any subset that hits <i>d</i> will do

Each answer is just *yes* or *no*. The promising function checks feasibility, and the search stops at the first valid configuration (or reports none). There is no notion of a *better* solution, hence no bound.

Branch and Bound is for problems that ask **both** questions — first *is this solution legal?* (feasibility) and then *is it the best legal one?* (optimality). The two examples in this chapter are exactly of that kind:

Problem	Feasibility criterion	Optimality criterion (the bound)
0/1 Knapsack	total weight of chosen items $\leq W$	<i>maximise</i> total value; bound = fractional-knapsack upper bound
TSP	tour visits every city exactly once and returns to start	<i>minimise</i> total tour length; bound = minimum-edge lower bound

Feasibility alone is not enough: a knapsack packing under the weight limit, or any complete tour, is *legal* but not necessarily *best*. The bound is precisely what lets us discard legal-but-suboptimal solutions that backtracking would have kept.

Could n -Queens ever be Branch and Bound?

Yes — the moment we attach an *optimisation* criterion to it. Suppose that, among all valid (non-attacking) placements, we also want the board where a **king is best protected** — say, the arrangement *maximising* the number of squares around the king that a queen defends. Now “no two queens attack” is the *feasibility* test (unchanged), and “most protected king” is the *optimality* test. We can compute an optimistic **upper bound** on the protection still achievable from a partial placement, prune any partial board whose bound cannot beat the best complete board found so far, and we have turned n -Queens into a genuine Branch-and-Bound problem. The lesson: feasibility makes a problem a backtracking problem; adding an objective to *optimise* is what makes it Branch and Bound.

4.3 0/1 Knapsack via Branch and Bound

Problem Statement (Levitin §3.4)

Given n items of known weights $w_1, w_2, \dots, w_n$ and values $v_1, v_2, \dots, v_n$ and a knapsack of capacity W , find the **most valuable subset** of the items that fits into the knapsack. In the 0/1 version each item is either taken whole or left behind (no fractions). Formally, choose $x_i \in \{0, 1\}$ to

$$\text{maximise } \sum_{i=1}^n v_i x_i \quad \text{subject to} \quad \sum_{i=1}^n w_i x_i \leq W.$$

Levitin introduces this problem in §3.4 (Exhaustive Search); here we solve it exactly with Branch and Bound.

Comparison: DP vs. Backtracking vs. B&B for 0/1 Knapsack

	DP	Backtracking	B&B
Approach	Fill $V[n][W]$ table	DFS, prune infeasible	Best-first, prune sub-optimal
Complexity	$\Theta(nW)$ <i>all cases</i> (input-insensitive)	worst $\Theta(2^n)$	worst $\Theta(2^n)$, typically $\ll$
Finds all?	Optimal value only	All optimal; no pruning for suboptimal	Optimal value; best-first finds it faster
Bound	Exact (table)	None (feasibility only)	Fractional relaxation (UB)

Algorithm

We give the **recursive** form first: it is the generic template (Algorithm BRANCHBOUND) instantiated, so the mapping to *Exit 1/2/3* is immediate. The upper bound is Levitin’s formula (12.1) — fill the remaining capacity $W - wt$ at the per-unit payoff v_{k+1}/w_{k+1} of the next item (see the *Fractional Optimisation* note below for why this fractional quantity is a valid bound):

$$\text{UB}(k, wt, val) = val + (W - wt) \frac{v_{k+1}}{w_{k+1}}, \quad \text{taking } \frac{v_{k+1}}{w_{k+1}} = 0 \text{ when no items remain.}$$

Algorithm 15: KnapsackBB-Rec(k, wt, val) — items sorted by v/w ; driver: $best \leftarrow 0$; KnapsackBB-Rec($0, 0, 0$)

```

1 if  $wt > W$  then return                                     // Exit 2: infeasible (overweight)
2 if  $val + (W - wt) v_{k+1} / w_{k+1} \leq best$  then return      // Exit 1: UB (12.1) cannot beat best
3 if  $k = n$  then  $best \leftarrow \max(best, val)$ ; return        // Exit 3: all items decided
4 KnapsackBB-Rec( $k+1, wt + w_{k+1}, val + v_{k+1}$ )              // branch: include item  $k+1$ 
5 KnapsackBB-Rec( $k+1, wt, val$ )                                // branch: exclude item  $k+1$ 

```

Here “not promising” specialises to *overweight* ($wt > W$); the bound is the fractional estimate (12.1). The $(W - wt) v_{k+1} / w_{k+1}$ term is exactly where a *fraction* of the next item enters — in the bound only, never in a stored solution. The two recursive calls are the include/exclude branch.

The same logic as the **iterative** best-first search actually run in practice — the recursion’s depth-first order is replaced by an explicit max-priority queue that expands the highest-UB node first:

Algorithm 16: KnapsackBB(*items* sorted by $v/w, W$)

```

1  $best \leftarrow 0$ 
2 Push root ( $k = 0, wt = 0, val = 0$ ) onto max-priority queue  $PQ$ 
3 while  $PQ \neq \emptyset$  do
4    $(k, wt, val) \leftarrow \text{EXTRACTMAX}(PQ)$  by UB
5   if  $k = n$  then
6      $best \leftarrow \max(best, val)$ ; continue
7   end
8   // Include item  $k + 1$ 
9   if  $wt + w_{k+1} \leq W$  then
10     $v' \leftarrow val + v_{k+1}; w' \leftarrow wt + w_{k+1}$ 
11     $UB_L \leftarrow v' + (W - w') v_{k+2} / w_{k+2}$  // formula (12.1)
12    if  $UB_L > best$  then push  $(k + 1, w', v')$  onto  $PQ$ 
13    end
14    // Exclude item  $k + 1$ 
15     $UB_R \leftarrow val + (W - wt) v_{k+2} / w_{k+2}$  // formula (12.1)
16    if  $UB_R > best$  then push  $(k + 1, wt, val)$  onto  $PQ$ 
17 end
18 return  $best$ 

```

Fractional Optimisation as the Bound

The bound deliberately solves an *easier* problem than the one we are actually solving. We are solving 0/1 knapsack ($x_i \in \{0, 1\}$), but to get a quick optimistic ceiling we *relax* it to the **continuous (fractional)** knapsack, allowing $0 \leq x_i \leq 1$. Since every 0/1 packing is also a legal fractional packing (each chosen item taken at fraction 1), the fractional optimum can only be larger:

$$\text{fractional optimum} \geq \text{0/1 optimum}.$$

So the relaxed value never *under*-estimates the true 0/1 best below a node — exactly the “optimistic, never rules out the real optimum” property a bound needs. Levitin states

this directly: “the optimal value of the solution to an instance of the continuous knapsack problem can serve as an upper bound on the optimal value of the discrete version.”

Trace Example (Levitin §12.2)

Items (already sorted by v/w):

Item	Weight w	Value v	v/w
1	4	40	10.0
2	7	42	6.0
3	5	25	5.0
4	3	12	4.0

Capacity $W = 10$.

UB computation (formula 12.1): at (weight w , value v , next item i), $UB = v + (W - w)(v_i/w_i)$ — fill the remaining capacity $W - w$ entirely at the next item’s per-unit payoff.

Node 0 (root): $wt = 0$, $val = 0$, next item 1.

$UB = 0 + (10 - 0) \times (40/4) = 10 \times 10 = \mathbf{100}$.

Expand Node 0: branch on item 1.

- **Node 1** (include item 1): $wt = 4$, $val = 40$, next item 2. $UB = 40 + (10 - 4) \times (42/7) = 40 + 36 = 76$. $76 > 0 = best \Rightarrow$ enqueue.
- **Node 2** (exclude item 1): $wt = 0$, $val = 0$, next item 2. $UB = 0 + (10 - 0) \times (42/7) = 60$. $60 > 0 \Rightarrow$ enqueue.

Expand Node 1 (UB=76, best in PQ): branch on item 2.

- **Node 3** (include items 1,2): $wt = 4 + 7 = 11 > 10 \Rightarrow$ **INFEASIBLE** (Exit 2).
- **Node 4** (include item 1, exclude item 2): $wt = 4$, $val = 40$, next item 3. $UB = 40 + (10 - 4) \times (25/5) = 40 + 30 = 70$. $70 > 0 \Rightarrow$ enqueue.

Expand Node 4 (UB=70, best in PQ): branch on item 3.

- **Node 5** (items 1,3): $wt = 9$, $val = 65$, next item 4. $UB = 65 + (10 - 9) \times (12/3) = 65 + 4 = 69$. Enqueue.
- **Node 6** (item 1, excl 2,3): $wt = 4$, $val = 40$, next item 4. $UB = 40 + (10 - 4) \times (12/3) = 40 + 24 = 64$. $64 > 0 \Rightarrow$ enqueue.

Expand Node 5 (UB=69, best): branch on item 4.

- **Node 7** (items 1,3,4): $wt = 9 + 3 = 12 > 10 \Rightarrow$ **INFEASIBLE** (Exit 2).
- **Node 8** (items 1,3, excl 4): $wt = 9$, $val = 65$. **LEAF**. $best \leftarrow \max(0, 65) = \mathbf{65}$ (Exit 3).

Check remaining nodes:

- Node 2: $UB = 60 < 65 = best \Rightarrow$ **PRUNE** (Exit 1).

- Node 6: $UB = 64 < 65 \Rightarrow$ **PRUNE** (Exit 1).

PQ is now empty. **Optimal value = 65** (items 1 and 3: $wt = 9 \leq 10$, $val = 40 + 25 = 65$).

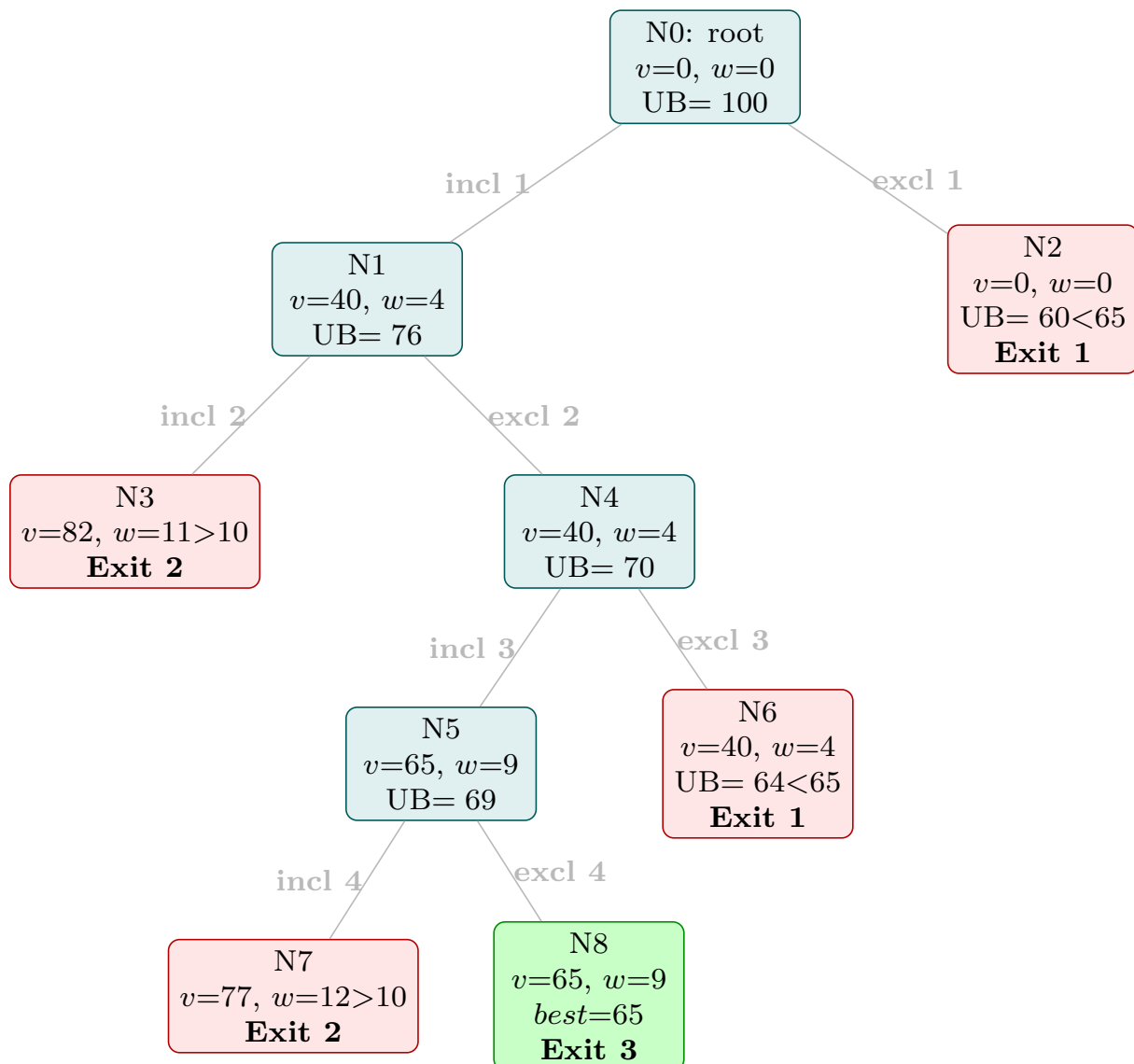


Figure 4.1: B&B state-space tree for 0/1 Knapsack (node ids match the trace above). **Green** = solution leaf (**Exit 3**, value 65); **Red** = terminated by **Exit 2** (infeasible/overweight) or **Exit 1** ($UB < best$). Bold edge labels give the include/exclude branch.

Complexity: 0/1 Knapsack B&B

Metric	Best	Average	Worst
Time	$\Theta(n)$	instance-dependent	$\Theta(2^n)$
Space	$\Theta(n)$	instance-dependent	$\Theta(2^n)$ (PQ size)

In the best case, a tight upper bound prunes all but one root-to-leaf path ($\Theta(n)$ nodes). In the worst case (bound always equals the true value), the whole tree is explored (2^n leaves), and such inputs are constructible — so the worst case is *tight*,

hence $\Theta(2^n)$, not merely $O(2^n)$. The average is genuinely instance-dependent: it swings with how tightly the bound prunes, so no single function bounds it both ways and we state no Θ . *Convention:* Θ is used wherever a case has a tight bound; descriptive text is used only where one does not.

4.4 Travelling Salesman Problem (B&B)

Real-World Applications

- **Logistics and delivery:** A courier optimises the order of parcel deliveries to minimise total distance driven.
- **Circuit board drilling:** A drill head must visit each hole exactly once; minimising path length reduces manufacturing time.
- **DNA sequencing:** Reconstruction of DNA sequences from overlapping fragments.
- **Vehicle routing:** Fleet scheduling for supply chains.

Problem Statement

Given n cities with a known cost $c(i, j)$ to travel from city i to city j , find a **Hamiltonian cycle** (visit each city exactly once and return to the start) of minimum total cost.

TSP is NP-hard: no polynomial-time algorithm is known. B&B finds the exact optimum but has exponential worst-case time.

Lower Bound Function

A valid lower bound for the TSP must be $\leq$ any complete tour. A simple lower bound:

$$\text{LB} = \frac{1}{2} \sum_{v \in V} (\text{cost of two cheapest edges incident to } v).$$

This is a valid lower bound because every vertex in any tour has exactly two incident edges.

Intuition. In *any* Hamiltonian tour each city is entered once and left once — exactly two incident edges. The cheapest those two edges could *possibly* be is the city's two smallest-cost edges. Summing this lower estimate over all cities counts every edge twice (once at each endpoint), so we halve it. The real tour can only use edges that are at least this cheap, therefore its cost cannot fall below the estimate — a valid lower bound. (We round *up* to the nearest integer when all costs are integers, since a tour cost is integral.)

Lower Bound for Partial Tours

When a partial tour has already fixed some edges, the bound is:

$$\begin{aligned} \text{LB}(\text{partial}) = & \text{cost of fixed edges} \\ & + \frac{1}{2} \sum_{v: \deg(v) < 2} (\text{two cheapest available edges at } v). \end{aligned}$$

Algorithm

The **recursive** form first, mapping onto the generic template (Algorithm BRANCHBOUND); TSP is a *minimisation*, so the bound is a lower bound and Exit 1 fires when $\text{LB} \geq \text{best}$:

Algorithm 17: TSP-BB-Rec(*path*, *cost*, *level*) — driver: *best* $\leftarrow$ a greedy tour; TSP-BB-Rec($\langle 1 \rangle$, 0, 1)

```

1 if not promising(path) then return                                // Exit 2: cannot complete a tour
2 if LB(path)  $\geq$  best then return                                // Exit 1: LB cannot beat best
3 if level = n then
4   | best  $\leftarrow$  min(best, cost + c(last(path), 1))           // Exit 3: close the tour
5   | return
6 end
7 foreach city c  $\notin$  path do
8   | TSP-BB-REC(path || c, cost + c(last(path), c), level+1) // branch
9 end

```

“not promising” rules out a partial path that cannot be closed into a tour; Exit 3 adds the return edge to city 1 and updates *best* with min.

The **iterative** best-first form actually run in practice — a min-priority queue keyed by the lower bound expands the most promising partial tour first. (Best-first needs the priority queue: recursion can only go depth-first, so expanding the globally most promising live node requires an explicit frontier.)

Algorithm 18: TSP-BranchBound($cost[1..n][1..n]$)

```

1 Compute initial LB for root using all cities
2  $best \leftarrow$  cost of some greedy tour (upper bound)
3  $PQ \leftarrow \{\text{root: partial tour starting at city 1}\}$ 
4 while  $PQ \neq \emptyset$  do
5    $node \leftarrow \text{EXTRACTMIN}(PQ)$  by LB
6   if  $LB(node) \geq best$  then continue // prune
7
8   if  $node$  is a complete tour then
9      $best \leftarrow \min(best, node.cost)$ ; continue
10  end
11  foreach city  $c$  not yet in  $node$  do
12    Form child by appending  $c$  to  $node$ 
13    Compute LB for child
14    if  $LB(child) < best$  then INSERT( $PQ$ , child)
15  end
16 end
17 return  $best$ 

```

Trace Example

Cities: $\{1, 2, 3, 4\}$. Cost matrix (symmetric, $c(i, i) = \infty$):

$$C = \begin{bmatrix} - & 3 & 6 & 7 \\ 3 & - & 2 & 8 \\ 6 & 2 & - & 4 \\ 7 & 8 & 4 & - \end{bmatrix}$$

Root lower bound: Two cheapest edges per vertex:

- City 1: 1-2 (3) + 1-3 (6) = 9
- City 2: 2-1 (3) + 2-3 (2) = 5
- City 3: 3-2 (2) + 3-4 (4) = 6
- City 4: 4-3 (4) + 4-1 (7) = 11

$LB(\text{root}) = (9 + 5 + 6 + 11)/2 = 15.5 \rightarrow 16$.

All tours from city 1 ($((4 - 1)!/2 = 3$ distinct tours for a symmetric graph):

Tour	Total cost
$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 1$	$3 + 2 + 4 + 7 = 16$
$1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 1$	$3 + 8 + 4 + 6 = 21$
$1 \rightarrow 3 \rightarrow 2 \rightarrow 4 \rightarrow 1$	$6 + 2 + 8 + 7 = 23$

B&B state-space tree (DFS from city 1; lower bounds drive pruning):

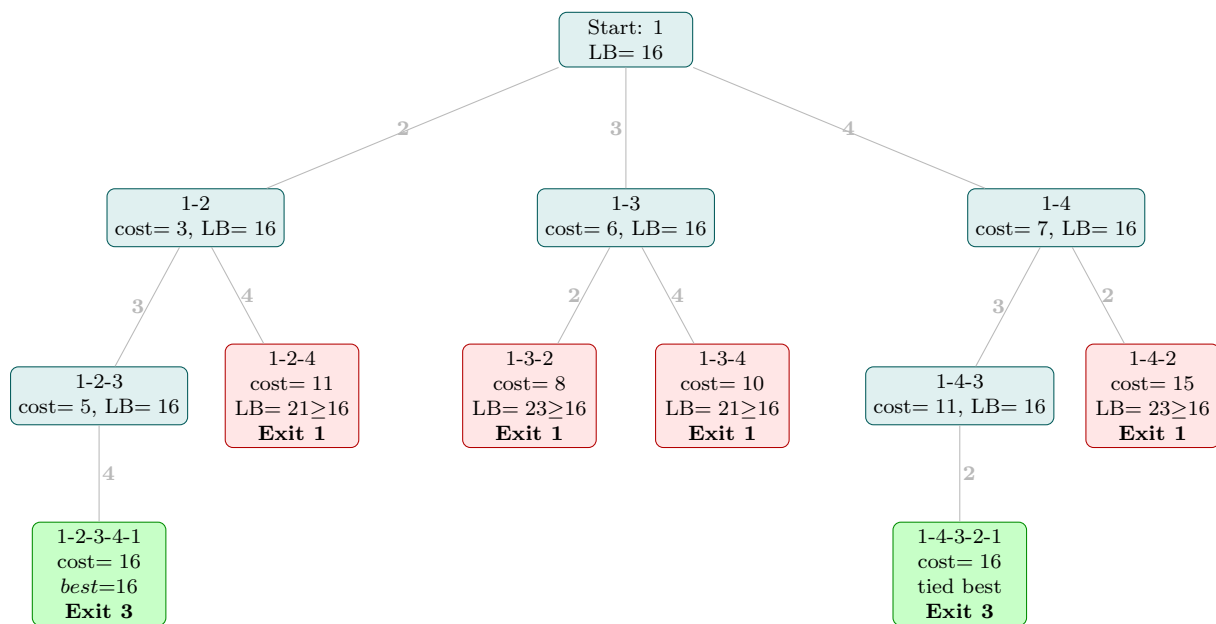


Figure 4.2: B&B state-space tree for 4-city TSP (DFS from city 1; node ids match the trace above). **Green** = complete tour (**Exit 3**); optimal cost 16, found at two tours (1→2→3→4→1 and 1→4→3→2→1). **Red** = terminated by **Exit 1** ($LB \geq best$). Bold edge labels give the next city added. (No **Exit 2** here: in a complete graph every partial path can be completed.)

Key step explained (pruning 1 → 2 → 4): After choosing the path 1 → 2 → 4 (cost = 3 + 8 = 11), the remaining edges must include 4 → 3 (cost 4) and 3 → 1 (cost 6) to complete the tour, giving partial $LB = 11 + 4 + 6 = 21$. Since $21 \geq 16$ (best known tour), this branch is pruned.

Complexity: TSP Branch and Bound

Metric	Best	Average	Worst
Time	$\Theta(n^2)$	instance-dependent	$\Theta(n!)$
Space	$\Theta(n)$	instance-dependent	$\Theta(n!)$ (PQ)

Same convention as the knapsack box: Θ where the case is tight (best prunes to one path; worst enumerates the full permutation tree, which is constructible), descriptive text for the instance-dependent average. **Without pruning:** $(n-1)!$ tours to enumerate from a fixed starting city. **With tight bounds:** B&B is the state-of-the-art exact solver for moderate n (e.g., the Concorde TSP solver has solved instances with thousands of cities). For large n , heuristics (nearest-neighbour, 2-opt) or approximation algorithms (Christofides: 3/2-approximation) are used instead.

4.5 End-of-Chapter Questions

Numerical Questions

1. Apply the Knapsack B&B algorithm to items (w, v) : $(2, 40), (3, 60), (4, 50)$, $W = 5$. Sort by v/w , compute UBs at each node, and determine the optimal solution.
2. For the same Knapsack instance, compare: (a) DP solution, (b) backtracking solution, (c) B&B solution. Which explores the fewest nodes? Why?
3. In the 4-city TSP trace, verify the lower bound $LB(\text{root}) = 16$ by showing the two cheapest edges incident to each city.
4. Rerun the TSP B&B on the cost matrix C but start with city 2 instead of city 1. Does the same optimal cost 16 emerge?
5. Modify the TSP cost matrix so that city 3 has very high outgoing costs ($c(3, i) = 100$ for all i). Show which branches are pruned immediately.
6. Explain why the B&B PQ for minimisation uses a min-priority queue on LB and for maximisation uses a max-priority queue on UB.
7. For the Knapsack B&B, why must items be *sorted by v/w* before the algorithm begins? What goes wrong if they are in arbitrary order?
8. Design a B&B bound function for the **assignment problem**: assign n workers to n jobs (one-to-one) to minimise total cost. (Hint: use the minimum cost in each row of the unassigned submatrix as a lower bound.)

LeetCode Practice

1. 1066 — Campus Bikes II (assignment problem, bitmask DP — compare with B&B)
2. 847 — Shortest Path Visiting All Nodes (TSP on small graphs, BFS)
3. 943 — Find the Shortest Superstring (TSP on strings, DP+bitmask)
4. 980 — Unique Paths III (grid TSP variant, backtracking)

Chapter 5

NP-Completeness

Chapter Outcomes

You will define the complexity classes P, NP, NP-hard, and NP-complete; explain the difference between *solving* and *verifying*; state Cook's Theorem; demonstrate polynomial-time reductions between NP-complete problems; and classify Clique and Vertex Cover as NP-complete.

5.1 Motivation: Why Some Problems Resist Efficient Algorithms

Throughout this course we have designed algorithms that run in $O(n \log n)$, $O(n^2)$, or even $O(n^3)$. For problems like Sorting ($\Theta(n \log n)$), MST ($O(E \log V)$), and Shortest Paths ($O(E \log V)$), we found polynomial-time solutions.

But for problems like:

- 0/1 Knapsack: DP gives $\Theta(nW)$ — pseudo-polynomial, not polynomial in input size.
- TSP: best known exact algorithm is $O(n^2 2^n)$ (Held-Karp DP) — exponential.
- n -Queens, Subset-Sum: backtracking with $O(2^n)$ or $O(n!)$ worst case.

No polynomial-time algorithm is known for these problems. NP-completeness theory explains *why*: these problems are believed to be inherently hard.

5.2 Complexity Classes P and NP

Decision Problems

NP-completeness is defined for **decision problems**: problems with a yes/no answer. Optimisation problems are converted to decision problems by asking “Is there a solution with cost $\leq k$?” (or $\geq k$ for maximisation).

Optimisation problem	Decision version
Find the shortest tour (TSP)	Does a tour of length $\leq L$ exist?
Find the maximum clique	Does a clique of size $\geq k$ exist?
Find the minimum vertex cover	Does a vertex cover of size $\leq k$ exist?

Class P

$\mathbf{P} = \{ \text{decision problems solvable in polynomial time } O(n^k) \text{ for some constant } k \}.$

Examples: Is this array sorted? Is this number prime? Is $u-v$ connected in this graph? Does an MST of weight $\leq W$ exist?

Nondeterministic Algorithms (the “N” in NP)

NP stands for *Nondeterministic Polynomial* time — *not* “non-polynomial”. To define it we first need the idea of a **nondeterministic algorithm**: an idealised (not physically realisable) algorithm that is allowed to *guess*. Following Horowitz & Sahni, it uses three primitives:

- **CHOICE**(S) — arbitrarily *guesses* and returns one element of S ;
- **SUCCESS** — halts and reports a successful (“yes”) completion;
- **FAILURE** — halts and reports an unsuccessful completion.

Semantics: the machine is assumed to always make choices that lead to **SUCCESS** *whenever some sequence of choices can*. So it “accepts” an input iff *some* sequence of guesses reaches **SUCCESS**; its running time is the length of that (shortest) guess-and-check path — guessing itself is treated as free.

Example 1 — nondeterministic search. Find a position of x in $A[1..n]$:

Algorithm 19: NDSearch($A[1..n], x$)

```

1  $j \leftarrow \text{CHOICE}(1, n)$                                 // guess an index
2 if  $A[j] = x$  then
3   | output  $j$ ; SUCCESS
4 else
5   | FAILURE
6 end
```

A deterministic search needs $O(n)$ comparisons; this version “succeeds” in $O(1)$ time — it guesses the right index (if one exists) and confirms it with a single comparison.

Example 2 — nondeterministic CNF-SAT. Decide whether $\phi(x_1, \dots, x_n)$ is satisfiable:

Algorithm 20: NDSat(ϕ)

```

1 for  $i \leftarrow 1$  to  $n$  do
2    $x_i \leftarrow \text{CHOICE}(0, 1)$                                 // guess each truth value
3 end
4 if  $\phi(x_1, \dots, x_n) = \text{true}$  then
5   SUCCESS
6 else
7   FAILURE
8 end

```

Guessing the n truth values costs $O(n)$ and evaluating ϕ costs $O(|\phi|)$ — *polynomial*. So a satisfiable formula is accepted by a polynomial-time nondeterministic algorithm.

This is exactly what NP means. The guesses (the CHOICE outputs) form a **certificate**, and the deterministic part after guessing is a **polynomial-time verifier**. The class **NP** (defined next) is precisely the problems with such a polynomial-time nondeterministic algorithm — equivalently, a polynomial-time-checkable certificate. Only once NP is pinned down do **NP-hard** and **NP-complete** become meaningful.

Class NP

NP = { decision problems where a “yes” answer can be *verified* in polynomial time }.

Certificate / witness: For a “yes” instance, a certificate is a short piece of evidence that allows efficient verification.

Problem	Certificate	Verification
0/1 Knapsack $\geq k$	A subset S of items	Check $\sum_{i \in S} v_i \geq k$ and $\sum_{i \in S} w_i \leq W$
TSP $\leq L$	A Hamiltonian cycle H	Check H visits all cities once and has cost $\leq L$
Clique of size k	A set C of k vertices	Check all pairs in C are connected
Subset-sum = d	A subset S	Check $\sum_{i \in S} w_i = d$

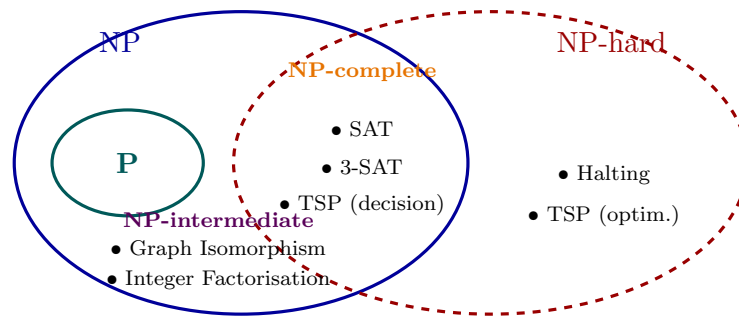
Key distinction:

- P: *solve* efficiently (find the answer from scratch).
- NP: *verify* a given answer efficiently (check if a proposed solution is correct).
- Every problem in P is also in NP (if you can solve it, you can certainly verify it).

P vs. NP: The Million-Dollar Question

$$\mathbf{P} \stackrel{?}{=} \mathbf{NP}$$

This is the most famous unsolved problem in computer science (Millennium Prize Problem). Most researchers believe $\mathbf{P} \neq \mathbf{NP}$, meaning that for some NP problems no polynomial-time solver exists. It has not been proved either way.



Assuming $P \neq NP$: **NP** and **NP-hard** *overlap*, and their intersection is exactly **NP-complete**. **P** lies inside **NP** (e.g. sorting, Dijkstra) and is *disjoint* from **NP-hard**. The part of **NP** lying outside both **P** and **NP-complete** holds the **NP-intermediate** problems (e.g. Graph Isomorphism, Integer Factorisation). Problems such as the Halting problem are **NP-hard** but *not* in **NP**.

Summary: P, NP, NP-Complete, NP-Hard

Region	Defining property	Examples
P (inner)	Deterministically <i>solvable</i> in polynomial time	sorting, Dijkstra
NP (blue)	<i>Verifiable</i> in polynomial time given a certificate — <i>solving</i> may still be exponential	all of P, plus 3-SAT, TSP (decision)
NP-complete (= $NP \cap NP\text{-hard}$)	In NP and every NP problem reduces to it	SAT, 3-SAT, TSP (decision)
NP-intermediate (exists if $P \neq NP$)	In NP , but believed <i>neither</i> in P nor NP-complete	Integer Factorisation, Graph Isomorphism, Discrete Logarithm
NP-hard	Every NP problem reduces to it (a “universal target”); need <i>not</i> be in NP	TSP (optimisation), Halting
Outside NP	Not even polynomial-time verifiable; may be unsolvable	Halting (undecidable)

Two things to remember:

- **NP = verifiable fast, not solvable fast.** Only the inner **P** region is “polynomial time”; the rest of **NP** is merely poly-time *checkable*.
- **NP-hard = relative hardness (reducibility), not a running-time label.** A problem can be **NP-hard** and exponential, or **NP-hard** and outright *undecidable* (the Halting problem).

5.3 NP-Completeness and Cook's Theorem

Polynomial-Time Reduction

Problem A **reduces to** problem B in polynomial time (written $A \leq_p B$) if: given any instance of A , we can transform it into an instance of B in polynomial time such that the answer to B gives the answer to A .

Consequence: If $B \in \mathbf{P}$ and $A \leq_p B$, then $A \in \mathbf{P}$. Contrapositively: if A is hard (not in $\mathbf{P}$) and $A \leq_p B$, then B is at least as hard as A .

NP-Complete and NP-Hard

- A problem X is **NP-hard** if every problem in NP reduces to X in polynomial time: $\forall L \in \mathbf{NP}, L \leq_p X$.
- A problem X is **NP-complete** if it is both NP-hard *and* in NP.

	NP-Hard	NP-Complete
In NP?	Not necessarily	Yes
Every NP problem reduces to it?	Yes	Yes
Examples	Halting problem, TSP optimisation	TSP decision, SAT, Clique

Boolean Formulas: Literals, CNF and DNF

A **Boolean variable** x_i is either **true (1)** or **false (0)**. Formulas use three connectives: **OR** ($\vee$), **AND** ($\wedge$), and **NOT** (written with an overbar, $\overline{x_i}$). A **literal** is a variable or its negation (x_i or $\overline{x_i}$) — the smallest unit that can be true or false.

Two building blocks:

- A **clause** is an *OR of literals*, e.g. $(x_1 \vee \overline{x_2} \vee x_3)$; it is true if *at least one* of its literals is true.
- A **term** is an *AND of literals*, e.g. $(x_1 \wedge \overline{x_2} \wedge x_3)$; it is true only if *every* literal is true.

Conjunctive Normal Form (CNF) — an *AND of clauses* (“product of sums”):

$$F = (x_1 \vee \overline{x_2} \vee x_3) \wedge (\overline{x_1} \vee x_2) \wedge (x_2 \vee \overline{x_3}).$$

F is true iff *every* clause is true — a list of simultaneous constraints.

Disjunctive Normal Form (DNF) — an *OR of terms* (“sum of products”):

$$G = (x_1 \wedge \overline{x_2}) \vee (\overline{x_1} \wedge x_3) \vee (x_2 \wedge \overline{x_3}).$$

G is true iff *at least one* term is fully true. Every Boolean function can be written in both forms (though converting between them may blow up exponentially in size).

The Satisfiability Problem (SAT)

Problem: SAT

Input: a Boolean formula ϕ in CNF.

Question: is there a truth assignment to the variables making $\phi = \text{true}$? If yes, ϕ is **satisfiable**; otherwise **unsatisfiable**.

Satisfiable example. $F = (x_1 \vee x_2 \vee x_3) \wedge (\overline{x_1} \vee \overline{x_2}) \wedge (x_2 \vee \overline{x_3})$. Try $x_1=1, x_2=0, x_3=0$: each clause evaluates to 1 $\Rightarrow$ **satisfiable**, witness $(1, 0, 0)$.

Unsatisfiable example. $(x_1) \wedge (\overline{x_1})$: no value of x_1 satisfies both.

Why CNF, not DNF? (an important subtlety)

Question	on DNF	on CNF
Satisfiable? (some assignment true)	Easy (P)	NP-complete
Tautology? (every assignment true)	co-NP-hard	Easy

DNF-satisfiability is easy: a DNF is satisfiable iff *some* term contains no variable together with its complement — just scan each term in $O(|\phi|)$. So “SAT is NP-complete” refers to **CNF-SAT**, where *all* clauses must hold at once; that simultaneous-constraint structure is exactly what makes it hard.

Cook’s Theorem (1971)

Theorem: SAT is NP-Complete (Cook, 1971)

The **Boolean Satisfiability Problem (SAT)** is NP-complete.

SAT: Given a Boolean formula ϕ in conjunctive normal form (CNF), does there exist a truth assignment to the variables that makes ϕ true?

Why this is the foundation: Cook showed that *every* problem in NP can be encoded as a SAT instance in polynomial time. Therefore, SAT is the “hardest” problem in NP. All subsequent NP-completeness proofs proceed by reducing from SAT (or a known NP-complete problem) to the new problem.

The NP-Completeness Proof Template

To show problem X is NP-complete:

1. **Show $X \in \text{NP}$:** describe a polynomial-time verifier for X .
2. **Show X is NP-hard:** find a known NP-complete problem Y (typically **3-SAT**) and show $Y \leq_p X$ (reduce Y to X in polynomial time).

3-SAT (the restricted form used for reductions)

3-SAT is SAT specialised to **3-CNF** — a CNF in which *every clause has exactly three literals*. For example,

$$\phi = (x_1 \vee \bar{x}_2 \vee x_3) \wedge (\bar{x}_1 \vee x_2 \vee \bar{x}_3) \wedge (x_1 \vee x_2 \vee x_3).$$

Any CNF can be turned into an equivalent 3-CNF in polynomial time by padding clauses shorter than three literals and splitting clauses longer than three using extra variables. Thus $\text{SAT} \leq_p \text{3-SAT}$, so 3-SAT is also NP-hard (and, being in NP, NP-complete).

A known NP-hard problem such as 3-SAT is then used to prove that a new problem X is NP-hard:

$$\text{SAT} \leq_p \text{3-SAT} \leq_p X.$$

The Clique problem (shown below) is proved NP-hard by reducing 3-SAT to Clique, and Clique is in turn reduced to Vertex Cover to prove Vertex Cover is also NP-hard. These three problems together show how a problem is proved to belong to a complexity class in the theory of computation.

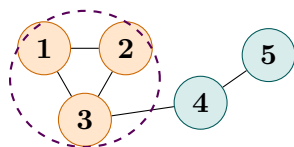
5.4 NP-Complete Problems: Clique & Vertex Cover

The Clique Problem

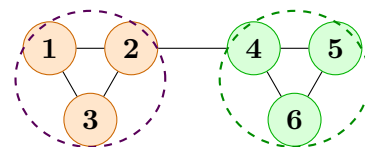
Problem: Clique

Input: An undirected graph $G = (V, E)$ and an integer k .

Question: Does G contain a **clique** of size k ? A clique is a subset $C \subseteq V$ of k vertices such that every pair is connected by an edge.



(a) Max clique $\{1, 2, 3\}$ (circled). Vertices 4, 5 *cannot* join it — 4 is adjacent only to 3 (and 5), so it is not connected to all of 1, 2, 3.



(b) Two vertex-disjoint cliques $\{1, 2, 3\}$ and $\{4, 5, 6\}$ (each circled). The bridge edge 2–4 does *not* merge them into a larger clique.

A clique need not contain every vertex (some vertices cannot belong), and a single graph may hold several disjoint cliques.

Fact: In a *complete* graph K_n (e.g. K_5 , where all $\binom{5}{2} = 10$ edges are present) *every* subset of vertices is a clique, since all pairs are adjacent. In a general graph, finding the largest clique is NP-hard.

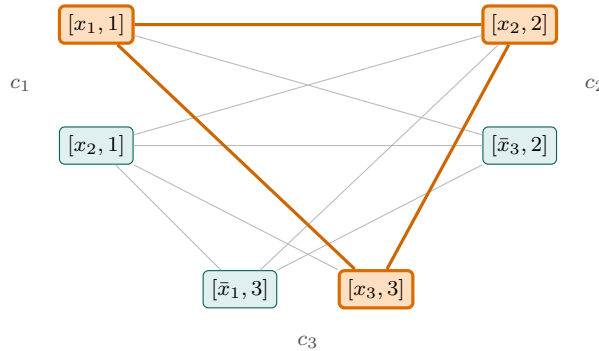
Verification (NP membership): Given a claimed clique C , check all $\binom{k}{2} = O(k^2)$ pairs; polynomial in k .

NP-hardness: $3\text{-SAT} \leq_p \text{Clique}$. Given a 3-CNF formula ϕ with m clauses and n variables, build a graph G where:

- For each literal in each clause, create a vertex: $3m$ vertices total.
- Connect two vertices with an edge if (a) they are in *different* clauses and (b) they are *not* complementary (x and $\bar{x}$ are not connected).

ϕ is satisfiable iff G has a clique of size m . The reduction runs in $O(m^2)$.

Worked example. Take $\phi = (x_1 \vee x_2) \wedge (x_2 \vee \bar{x}_3) \wedge (\bar{x}_1 \vee x_3)$ ($m = 3$ clauses). Make one vertex per literal occurrence and join two vertices when they are in *different* clauses and *not* complementary. The only forbidden cross-clause pairs are the complementary ones $[x_1, 1] - [\bar{x}_1, 3]$ and $[\bar{x}_3, 2] - [x_3, 3]$.



Reduction graph for ϕ . Orange vertices/edges = a clique of size $m = 3$ ($[x_1, 1], [x_2, 2], [x_3, 3]$ — one per clause). Grey edges are the other legal connections.

The clique $\{[x_1, 1], [x_2, 2], [x_3, 3]\}$ takes one vertex per clause with no complementary pair, so setting each chosen literal true gives the consistent assignment $x_1=1, x_2=1, x_3=1$:

$$(x_1 \vee x_2) \wedge (x_2 \vee \bar{x}_3) \wedge (\bar{x}_1 \vee x_3) = (1) \wedge (1) \wedge (1) = 1.$$

Why it is a *maximum* clique. Two vertices of the *same* clause are never joined, so any clique has *at most one vertex per clause* — the largest possible clique is therefore size m . Hence the graph has a clique of size m iff ϕ is satisfiable: deciding MAX-CLIQUE on this constructed graph *is* deciding 3-SAT.

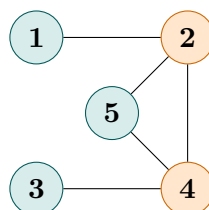
The Vertex Cover Problem

Problem: Vertex Cover

Input: An undirected graph $G = (V, E)$ and an integer k .

Question: Does G have a **vertex cover** of size k ? A vertex cover is a subset $S \subseteq V$ such that every edge has at least one endpoint in S .

Example:



$\{2, 4\}$ (orange nodes) is a vertex cover of size 2: every one of the edges $(1, 2)$, $(2, 4)$, $(3, 4)$, $(4, 5)$, $(2, 5)$ has at least one endpoint in $\{2, 4\}$. It is also a *minimum* cover — no single vertex covers all five edges.

Verification (NP membership): Given S , check every edge (u, v) : at least one of u, v is in S . $O(|E|)$ time — polynomial.

Finding a Vertex Cover — Greedy Algorithm

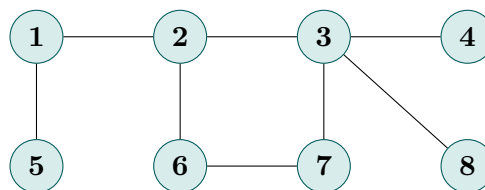
A simple greedy method repeatedly takes the vertex that covers the most still-uncovered edges:

Algorithm 21: VertexCover-Greedy(G)

```

1  $S \leftarrow \emptyset$ 
2 while  $G$  still has at least one edge do
3   pick a vertex  $v$  of maximum degree
4    $S \leftarrow S \cup \{v\}$ 
5   remove  $v$  and all edges incident to  $v$ ; update the degrees
6 end
7 return  $S$ 
```

Worked example. Find a vertex cover of the graph below.



Initial degrees: 1:2, 2:3, 3:4, 4:1, 5:1, 6:2, 7:2, 8:1 (8 edges in all). The greedy steps:

Step	Max-degree vertex	Cover S	Edges removed	Edges still uncovered
1	3 (deg 4)	$\{3\}$	2-3, 3-4, 3-7, 3-8	1-2, 1-5, 2-6, 6-7
2	6 (deg 2, tie 1/2/6)	$\{3, 6\}$	2-6, 6-7	1-2, 1-5
3	1 (deg 2)	$\{3, 6, 1\}$	1-2, 1-5	— (none)

Result. Vertex cover $\{1, 3, 6\}$ (size 3); it covers all eight edges. (At step 2 the tie among 1, 2, 6 could be broken differently, giving another valid cover.)

Note. This greedy “maximum-degree” rule returns a *valid* cover, but not always a *minimum* one — finding the *minimum* vertex cover is the NP-hard part. (Here it happens to be minimum, since vertex 3’s four edges force any cover to include 3 and then two more vertices.)

Key relationship — Clique and Vertex Cover are complementary:

Theorem: $\text{Clique} \leq_p \text{Vertex Cover}$

G has a clique of size $k \iff$ the complement graph $\bar{G}$ has a vertex cover of size $|V| - k$.

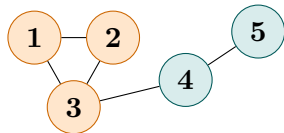
Complement graph $\bar{G}$: same vertex set; an edge (u, v) exists in $\bar{G}$ iff it does *not* exist in G .

Reduction: Given a Clique instance (G, k) , output the Vertex Cover instance $(\bar{G}, |V| - k)$. Computing $\bar{G}$ takes $O(V^2)$ — polynomial.

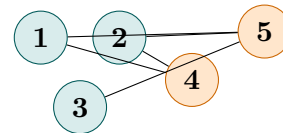
Proof sketch: If C is a clique of size k in G , then $V \setminus C$ has size $|V| - k$. For any edge (u, v) in $\bar{G}$, the edge (u, v) is absent from G , so at least one of u, v is not in C , meaning at least one is in $V \setminus C$. Thus $V \setminus C$ is a vertex cover of $\bar{G}$ of size $|V| - k$. $\square$

In words. Inside the clique C every pair is an edge of G , so *no* edge of $\bar{G}$ lies between two vertices of C — in $\bar{G}$, the clique C is an *independent set* (no edges among its vertices). Every edge of $\bar{G}$ must therefore have at least one endpoint *outside* C , i.e. in $V \setminus C$. Hence $V \setminus C$ touches every edge of $\bar{G}$, so it is a vertex cover. (Recall: a vertex cover is a set of *vertices*, not edges.)

Worked illustration (the clique graph used earlier): $V = \{1, 2, 3, 4, 5\}$ with $E(G) = \{1-2, 2-3, 1-3, 3-4, 4-5\}$ and clique $C = \{1, 2, 3\}$ ($k = 3$). The complement keeps the vertices and flips every pair, so its edges are the *non*-edges of G : $\bar{E} = \{1-4, 1-5, 2-4, 2-5, 3-5\}$.



(a) G — clique $C = \{1, 2, 3\}$ (orange).



(b) $\bar{G}$ — vertex cover $V \setminus C = \{4, 5\}$ (orange).

In $\bar{G}$ every edge — 1-4, 2-4 (via 4) and 1-5, 2-5, 3-5 (via 5) — has an endpoint in $\{4, 5\}$. So $\{4, 5\}$, of size $|V| - k = 2$, is a vertex cover of $\bar{G}$, exactly as the theorem promises. Note the clique vertices $\{1, 2, 3\}$ form an *independent set* in $\bar{G}$ (no edge among them), which is why the *other* two vertices suffice to cover everything.

The NP-Complete Family (Summary)

Problem	Type	Proved NP-complete by
SAT	Satisfiability	Cook (1971) — original
3-SAT	Restricted satisfiability	Karp (1972), from SAT
Clique	Graph	Karp (1972), from 3-SAT
Independent Set	Graph	Karp (1972), from Clique
Vertex Cover	Graph	Karp (1972), from Clique/IS
Hamiltonian Path	Graph	Karp (1972)
TSP (decision)	Optimisation	From Hamiltonian Path
Subset Sum	Number	From 3-SAT
0/1 Knapsack	Optimisation	From Subset Sum
3-Coloring	Graph	From 3-SAT

5.5 End-of-Chapter Questions

Conceptual and Proof Questions

1. Define P, NP, NP-hard, and NP-complete in your own words. Give one example of each category.
2. If $\mathbf{P} = \mathbf{NP}$, what would this imply for cryptography? (Hint: public-key cryptography relies on the hardness of integer factorisation.)
3. The Subset-Sum decision problem: given S and d , does a subset summing to d exist? Show it is in NP by describing a polynomial-time verifier.
4. Prove that if Clique has a polynomial-time algorithm, then Vertex Cover also has one. (Use the $\text{Clique} \leq_p \text{Vertex Cover}$ reduction.)
5. 3-SAT formula: $(\bar{x}_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \bar{x}_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \bar{x}_3)$. Is it satisfiable? Find a satisfying assignment or prove none exists.
6. For the graph in the Vertex Cover example (Section 4.4), find all vertex covers of size 2. Is $\{1, 4\}$ also a vertex cover? Why or why not?
7. Explain the difference between NP-hard and NP-complete using the Halting Problem as an example of a problem that is NP-hard but *not* in NP.
8. The Independent Set problem: given G and k , does G contain an independent set of size k ? Show it is NP-complete by reduction from Clique.

LeetCode and Related Practice

1. 1306 — Jump Game III (reachability — in P)
2. 139 — Word Break (DP — polynomial)
3. 131 — Palindrome Partitioning (backtracking — NP-like exhaustive)
4. Study: Why is 0/1 Knapsack NP-complete (decision version)? Why does the

DP solution not contradict this?

5. Study: Describe a 2-approximation algorithm for Vertex Cover (repeat: pick any uncovered edge, add both endpoints, repeat).

Note: Two Routes to a Vertex Cover (and a Common Trap)

For the clique graph $V = \{1, 2, 3, 4, 5\}$, $E(G) = \{1-2, 2-3, 1-3, 3-4, 4-5\}$ with clique $C = \{1, 2, 3\}$, there are two ways to obtain a vertex cover — and they apply to *different graphs*, which is exactly where students slip.

1. **Complement of the clique.** The reduction $\text{Clique} \leq_p \text{Vertex Cover}$ gives a cover of the *complement* graph $\bar{G} = (V, \bar{E})$, where $\bar{E} = \{1-4, 1-5, 2-4, 2-5, 3-5\}$:

$$\text{VC}(\bar{G}) = V \setminus C = \{4, 5\}, \quad \text{size } |V| - k = 5 - 3 = 2.$$

2. **Greedy max-degree** (Algorithm 21). To compare like with like, run it on *the same graph* $\bar{G}$ for which $\{4, 5\}$ is claimed to be a cover. Degrees in $\bar{G}$: 1:2, 2:2, 3:1, 4:2, 5:3.

Step	Max-degree vertex	Cover S	Edges removed	Edges still uncovered
1	5 (deg 3)	$\{5\}$	1-5, 2-5, 3-5	1-4, 2-4
2	4 (deg 2)	$\{5, 4\}$	1-4, 2-4	— (none)

Greedy returns $\{4, 5\}$ — **identical** to the complement cover $V \setminus C$, and minimum for $\bar{G}$ (no single vertex hits all five edges of $\bar{E}$).

The trap: the two routes target different graphs

- $\{4, 5\}$ is a cover of $\bar{G}$, *not* of G : in G the edge 1-2 has neither endpoint in $\{4, 5\}$. Running Algorithm 21 on G itself (degrees 1:2, 2:2, 3:3, 4:2, 5:1) starts with vertex 3 and yields a *size-3* cover such as $\{1, 3, 4\}$ — a different object answering a different question.
- The agreement here is not guaranteed in general: greedy max-degree can return a *non-minimum* cover, while the complement route returns exactly $V \setminus C$ (minimum iff C is a *maximum* clique). They coincided only because $\{4, 5\}$ is simultaneously the greedy output and the minimum cover of $\bar{G}$.

Appendix A

Appendix: Core Theorems and Algorithm Reference

Appendix Purpose

This appendix consolidates theorem statements used in CIE-3 analysis, together with correctness arguments and a complexity summary for all major algorithms covered.

A.1 Core Theorems

Theorem G1: MST Cut Property

Statement. For any cut $(S, V \setminus S)$ of a connected weighted undirected graph G with distinct edge weights, the minimum-weight edge crossing the cut belongs to every MST of G .

Proof sketch. Let $e^* = (u, v)$ be the minimum crossing edge with $u \in S, v \notin S$. Suppose T is an MST that does not contain e^* . Since T is a spanning tree, it has a unique path P from u to v . P must cross the cut at some edge $e' \neq e^*$. Form $T' = T - e' + e^*$. Then T' is still spanning (connected) and has weight $w(T') = w(T) - w(e') + w(e^*) < w(T)$ (since e^* is the minimum crossing edge), contradicting that T is a minimum spanning tree. $\square$

Use: Justifies Prim's greedy choice at each step.

Theorem G2: MST Cycle Property

Statement. For any cycle C in G with distinct edge weights, the maximum-weight edge in C does not belong to any MST.

Proof sketch. Let $e = (u, v)$ be the maximum-weight edge in cycle C . Suppose T is an MST containing e . Removing e splits T into two components; the other edges of C form a path between the two components. Let e' be any other edge of C that crosses this cut. Then e' has smaller weight than e , so $T' = T - e + e'$ has smaller total weight, contradicting that T is an MST. $\square$

Use: Justifies Kruskal's cycle-rejection step.

Theorem G3: Greedy Choice for Fractional Knapsack (Exchange Argument)

Statement. The greedy algorithm that always selects the item with the highest remaining value density v_i/w_i yields an optimal solution to the Fractional Knapsack problem.

Proof sketch (exchange argument). Suppose an optimal solution OPT differs from the greedy solution G by taking less of item i (density ρ_i) and more of item j (density $\rho_j < \rho_i$). Swap a small amount $\varepsilon > 0$ of item j for the same weight of item i . Since $\rho_i > \rho_j$, the total value increases by $\varepsilon(\rho_i - \rho_j) > 0$. This contradicts the optimality of OPT. $\square$

Theorem G4: Huffman Coding Optimality (Sibling Property)

Statement. There exists an optimal prefix-free code tree in which the two symbols with the smallest frequencies are siblings (have the same parent) at the deepest level.

Proof sketch. Let x and y be the two least-frequent symbols. In any optimal tree, the two deepest leaves (longest codes) can be swapped with x and y without increasing the WPL, because x and y have the lowest frequencies (longer codes hurt less). This shows that an optimal tree can be restructured so that x and y are siblings at the deepest level, proving the greedy merge step is safe. $\square$

Corollary. The Huffman algorithm produces an optimal prefix-free code tree (Huffman, 1952).

Theorem G5: Dijkstra's Correctness (Relaxation Invariant)

Statement. When a vertex u is extracted from the priority queue in Dijkstra's algorithm, $d[u]$ equals the true shortest-path distance $\delta(s, u)$.

Proof sketch (by induction on settled vertices). *Base:* $d[s] = 0 = \delta(s, s)$. $\checkmark$

Inductive step: Suppose all previously settled vertices $u_1, \dots, u_{k-1}$ have correct distances. Let u_k be the next vertex extracted (smallest d value). Assume for contradiction that $d[u_k] > \delta(s, u_k)$. Then there is a shorter path $P : s \rightarrow \dots \rightarrow u_k$. Let y be the first unsettled vertex on P and x be its settled predecessor. By the inductive hypothesis, $d[x] = \delta(s, x)$. Since edge (x, y) was relaxed when x was settled: $d[y] \leq d[x] + w(x, y) = \delta(s, x) + w(x, y) \leq \delta(s, y) \leq \delta(s, u_k)$. But this means $d[y] \leq d[u_k]$, so u_k would not have been extracted before y — contradiction. $\square$

Use: Guarantees Dijkstra's greedy choice (settling the minimum- d vertex) is correct when all edge weights are non-negative.

Theorem B1: Backtracking Completeness

Statement. A backtracking algorithm with a correct promising function finds *all* solutions to the problem.

Proof sketch. Backtracking traverses every node of the state-space tree that is not pruned. A node is pruned only if the promising function returns false. If the promising

function is correct (returns false only when no solution lies in the subtree), no valid leaf is pruned. Therefore, every complete solution is eventually reached. $\square$

Theorem B2: B&B Optimality

Statement. Branch and Bound with a valid bounding function returns an optimal solution.

Proof sketch. A node is pruned only when its bound (UB for max, LB for min) is no better than the current best solution. If the bounding function is valid (UB $\geq$ actual optimal of any completion, or LB $\leq$ actual cost of any completion), then no subtree containing a better solution is pruned. The algorithm therefore explores every subtree that could contain a better solution and correctly identifies the optimum. $\square$

Theorem NP1: Cook's Theorem (SAT is NP-Complete)

Statement (Cook, 1971). The Boolean Satisfiability Problem (SAT) is NP-complete.

Significance. This was the first problem proved NP-complete. Every subsequent NP-completeness proof (Karp's 21 problems, 1972, and thousands since) proceeds by polynomial reduction from SAT or a known NP-complete problem.

Proof idea. Any problem $L \in \mathbf{NP}$ is decided by a non-deterministic polynomial-time Turing machine M on input x . The computation of M on x can be encoded as a Boolean formula $\phi_{M,x}$ in CNF such that $\phi_{M,x}$ is satisfiable iff M accepts x . The encoding is polynomial in $|x|$. $\square$

Theorem NP2: Clique $\leq_p$ Vertex Cover

Statement. $G = (V, E)$ has a clique of size k if and only if the complement graph $\bar{G} = (V, \bar{E})$ has a vertex cover of size $|V| - k$.

Proof. ($\Rightarrow$) Let $C \subseteq V$ with $|C| = k$ be a clique in G . Let $S = V \setminus C$ ($|S| = |V| - k$). Take any edge $(u, v) \in \bar{E}$ (i.e., $(u, v) \notin E$). Since C is a clique, both u and v cannot be in C ; so at least one of u, v is in S . Thus S covers every edge of $\bar{G}$: S is a vertex cover of $\bar{G}$.

($\Leftarrow$) Let S be a vertex cover of $\bar{G}$ with $|S| = |V| - k$. Let $C = V \setminus S$ ($|C| = k$). For any two vertices $u, v \in C$, the pair $(u, v) \notin \bar{E}$ (otherwise S would need to cover it, but neither u nor v is in S). Hence $(u, v) \in E$, and C is a clique in G . $\square$

A.2 Complexity Summary: All CIE-3 Algorithms

Algorithm	Best	Average	Worst	Space
Fractional Knapsack	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$O(1)$
Prim's (binary heap)	$O((V+E) \log V)$	$O((V+E) \log V)$	$O((V+E) \log V)$	$O(V)$
Kruskal's	$\Theta(E \log E)$	$\Theta(E \log E)$	$\Theta(E \log E)$	$O(V)$
Dijkstra's (binary heap)	$O((V+E) \log V)$	$O((V+E) \log V)$	$O((V+E) \log V)$	$O(V)$
Huffman Coding	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n)$
n -Queens	$O(n)$	—	$O(n!)$	$\Theta(n)$
Subset-Sum (backtracking)	$O(1)$	—	$O(2^n)$	$\Theta(n)$
Knapsack B&B	$O(n)$	—	$O(2^n)$	$O(2^n)$
TSP B&B	$O(n^2)$	—	$O(n!)$	$O(n!)$
Horspool's	$\Theta(n)$	$\Theta(n/m)$	$O(nm)$	$\Theta(\Sigma)$
Boyer-Moore	$\Theta(n/m)$	$\Theta(n/m)$	$O(nm)$	$\Theta(m+ \Sigma)$
Checking $X \in \mathbf{NP}$	$O(\text{poly})$	—	$O(\text{poly})$	—

A.3 Comparison of Paradigms: CIE-3

Paradigm	Goal	State-space	Pruning / speedup
Space-time trade-off	Fast search/lookup	No tree	Precomputed shift/index table
Greedy	Optimisation (one pass)	No explicit tree	Greedy-choice property
Backtracking	Constraint satisfaction	DFS tree	Feasibility check
Branch & Bound	Optimisation (exact)	Best-first tree	Bounding function vs. best
NP-completeness	Classification only	—	Polynomial reductions

A.4 Supplementary: Reductions & Extra Examples (beyond the CIE-3 syllabus)

The material below extends Chapter 5. It is **not** part of the CIE-3 syllabus (which covers only the P / NP / NP -hard / NP -complete differences, Satisfiability, the Clique problem, and the Vertex Cover problem), but it is useful for a deeper understanding of polynomial-time reductions.

What “ $\leq_p$ ” does and does not constrain

The “ p ” in $L \leq_p X$ bounds *only the transformation* — the translator f that maps an instance of L to an instance of X must run in **polynomial time**. It says *nothing* about how hard X is to *solve*:

$$\underbrace{L \xrightarrow{f} X}_{\text{reduction: polynomial}} \qquad \underbrace{\text{solve } X}_{\text{unconstrained (may be exponential, or undecidable)}}$$

Consequences:

- NP-hardness is a **lower bound** (“ X is *at least as hard* as all of NP”), never an upper bound — solving X may take exponential time or worse.
- X need *not* be in P. Indeed, if an NP-hard X *were* in P then $\mathbf{P} = \mathbf{NP}$ (translate any L in poly time, then solve X in poly time).
- X need not even be in NP or be *decidable*: the Halting problem is NP-hard yet unsolvable, and TSP-optimisation is NP-hard yet not in NP.

A polynomial reduction therefore carries *easiness downward* (X easy $\Rightarrow L$ easy) and *hardness upward* (L hard $\Rightarrow X$ hard): the translator is cheap, but what it points at can be arbitrarily expensive.

Worked Reduction: Hamiltonian Cycle $\leq_p$ TSP

We *solve* the Hamiltonian-Cycle problem by transforming it into a Travelling-Salesman instance and calling a TSP solver.

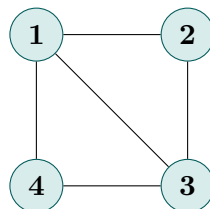
- **Problem A (Hamiltonian Cycle, HC):** does graph $G = (V, E)$ contain a cycle that visits every vertex exactly once?
- **Problem B (TSP, decision):** given a complete weighted graph and a bound L , is there a tour of total cost $\leq L$?

The transformation f . Given G with n vertices, build the *complete* graph on the same vertices with

$$\text{cost}(i, j) = \begin{cases} 1 & \text{if } (i, j) \in E, \\ 2 & \text{if } (i, j) \notin E, \end{cases} \quad \text{and set } L = n.$$

Constructing this cost matrix takes $O(n^2)$ time — polynomial.

A concrete instance. Let G be the graph below ($n = 4$):



so $E = \{1-2, 2-3, 3-4, 4-1, 1-3\}$ and the only non-edge is 2-4. Applying f (cost 1 for edges,

2 for the non-edge, $L = 4$):

	1	2	3	4
1	—	1	1	1
2	1	—	1	2
3	1	1	—	1
4	1	2	1	—

Solve B , read off A . A tour of cost ≤ 4 is $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 1$ (cost $1+1+1+1 = 4$). Stripping the weights, that tour uses only edges of G , so it *is* a Hamiltonian cycle. Conversely, a tour of cost $\leq n$ must use n cost-1 edges, all in E , hence a Hamiltonian cycle. Thus $\text{HC}(G) = \text{TSP}(f(G), n)$: this is $\text{HC} \leq_p \text{TSP}$, and (HC being NP-complete) it proves **TSP is NP-hard**.

Two Uses of Reduction (same relation, opposite goals)

The relation $A \leq_p B$ always reads “ A is no harder than B ,” but it is used in two opposite ways:

	Upper-bound use (algorithm design)	Lower-bound use (hardness)
Reduce...	<i>your</i> problem $\rightarrow$ a <i>known-easy</i> B	a <i>known-hard</i> $A \rightarrow$ <i>your</i> problem
If...	B is easy and $A \leq_p B$	A is hard and $A \leq_p B$
then...	A is easy too	B is hard too
Example	Two-Sum $\leq_p$ set-membership	HC $\leq_p$ TSP (above)

Upper-bound example — Two-Sum is really linear, not quadratic. “Is there a pair with $A[i] + A[j] = t$?” looks $O(n^2)$ (test all pairs). But it is the same as asking, for each $x \in A$, whether $t - x$ is *also present* — a sequence of **set-membership** queries, which a hash set answers in $O(1)$ each:

build a hash set while scanning A ; for each x , if $(t - x)$ is in the set, answer “yes”. This reduces Two-Sum to set-membership (a known $O(n)$ problem), *proving* Two-Sum is $O(n)$ — the quadratic double loop was just a naive formulation, not the problem’s true difficulty.

Same machinery, opposite direction to $\text{HC} \leq_p \text{TSP}$ above: there we reduced a *hard* problem *into* TSP to show TSP is hard; here we reduce Two-Sum *into* an *easy* problem to show Two-Sum is easy.

A String-Selection Problem is CNF-SAT

Problem (Karp). Given several sequences of upper- and lowercase letters, pick one letter from each sequence *without* ever choosing both the upper- and lowercase version of any letter. (E.g. from Abc , BC , aB , ac one can pick A,B,B,c ; but AB , Ab , aB , ab admits no valid choice.) This is *exactly* CNF-SAT, under the dictionary:

String problem	Boolean formula
a letter, e.g. A/a	a variable a
uppercase A	positive literal a (set $a = \text{true}$)
lowercase a	negative literal $\bar{a}$ (set $a = \text{false}$)
a sequence of letters	a clause (OR of those literals)
selecting one letter from a sequence	making ≥ 1 literal of that clause true (clause satisfied)
never pick both A and a	a consistent assignment (a not both true and false)
a valid selection from <i>all</i> sequences	the conjunction of all clauses is satisfiable

Instance 1 (satisfiable): sequences Abc , BC , aB , ac become

$$F_1 = (a \vee \bar{b} \vee \bar{c}) \wedge (b \vee c) \wedge (\bar{a} \vee b) \wedge (\bar{a} \vee \bar{c}).$$

The selection A,B,B,c is the assignment $a=1, b=1, c=0$, which makes every clause true.

Instance 2 (unsatisfiable): sequences AB , Ab , aB , ab become

$$F_2 = (a \vee b) \wedge (a \vee \bar{b}) \wedge (\bar{a} \vee b) \wedge (\bar{a} \vee \bar{b}),$$

all four 2-literal clauses over $\{a, b\}$ — the smallest unsatisfiable CNF, so “no selection is possible.” Because a sequence may hold three or more letters, this is general CNF-SAT, hence the string-selection problem is **NP-complete**.

Independent Set (related NP-complete problem)

A set $I \subseteq V$ is an **independent set** if no two vertices in I are adjacent. **Relationship:** I is an independent set in G iff I is a clique in $\bar{G}$, and $V \setminus I$ is a vertex cover in G . So all three problems (Clique, Independent Set, Vertex Cover) are NP-complete and mutually reducible.

Dealing with NP-Complete Problems in Practice

When a problem is NP-complete, several practical strategies exist:

1. **Exact algorithms for small n :** B&B, backtracking — exponential but fast enough for $n \leq 20\text{--}30$.
2. **Approximation algorithms:** provable performance guarantees — e.g. 2-approximation for Vertex Cover; $3/2$ -approximation (Christofides) for TSP.
3. **Heuristics:** greedy initialisation + local search (2-opt for TSP), simulated annealing, genetic algorithms.
4. **Parameterised algorithms:** if the input has bounded treewidth or special structure, exact polynomial or FPT algorithms may exist.
5. **Pseudo-polynomial algorithms:** DP for Knapsack is polynomial in W but not in $\log W$.

A.5 Extra Question Bank

Proof and Analysis Questions

1. Prove that Dijkstra's algorithm fails on the graph with edge $s \rightarrow a : 4$, $s \rightarrow b : 5$, $b \rightarrow a : -3$. What is the correct shortest distance to a ? Which algorithm should be used instead?
2. Use the Cut Property to prove that Prim's algorithm always produces a minimum spanning tree.
3. Use the Cycle Property to prove Kruskal's algorithm always produces an MST.
4. Prove that the Huffman tree for $n = 2$ symbols gives each symbol a 1-bit code, regardless of frequency.
5. Show that Subset-Sum is reducible to 0/1 Knapsack in polynomial time. (Hint: set $v_i = w_i$ and $W = d = k$.)
6. Show that 3-SAT is in NP by describing the certificate and verifier.
7. Describe the independent set problem and prove it is NP-complete by reduction from Clique.
8. Explain why a pseudo-polynomial algorithm for Knapsack does not imply $\mathbf{P} = \mathbf{NP}$.

LeetCode Practice (Mixed)

1. 1584 — Min Cost to Connect All Points (Kruskal's / Prim's)
2. 743 — Network Delay Time (Dijkstra's)
3. 1167 — Min Cost to Connect Sticks (Huffman)
4. 51 — N-Queens (backtracking)
5. 698 — Partition to K Equal Sum Subsets (backtracking)
6. 322 — Coin Change (DP, compare with backtracking)
7. 416 — Partition Equal Subset Sum (DP version of subset-sum)
8. 1269 — Number of Ways to Stay in the Same Place After Some Steps (DP)

Bibliography

- [1] A. Levitin, *Introduction to the Design and Analysis of Algorithms*, 3rd ed., Pearson, 2012.
- [2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed., MIT Press, 2022.